

SVEUČILIŠTE U ZAGREBU  
**FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA**

DIPLOMSKI RAD br. 1586

**IMPLEMENTACIJA SREDIŠNJEG SUSTAVA ZA  
NADZOR I VIZUALIZACIJU KOMUNIKACIJSKIH  
OBRAZACA LOKALNE MREŽE**

Ante Čavar

Zagreb, lipanj, 2026.

## **DIPLOMSKI ZADATAK br. 1586**

Pristupnik: **Ante Čavar (0036540817)**  
Studij: Računarstvo  
Profil: Računarska znanost  
Mentor: izv. prof. dr. sc. Stjepan Groš

Zadatak: **Implementacija središnjeg sustava za nadzor i vizualizaciju komunikacijskih obrazaca lokalne mreže**

### Opis zadatka:

S porastom broja povezanih uređaja u lokalnim mrežama, vrlo je teško uočiti neželjeni ili maliciozni mrežni promet koji potiče iz lokalne mreže. Standardna mrežna oprema često ne pruža dovoljnu vidljivost u specifičnosti dolaznih i odlaznih mrežnih konekcija, zbog čega je otežano utvrditi s kojim točno vanjskim resursima komuniciraju lokalni uređaji. Detekcija takvog mrežnog prometa ključna je otkrivanje sigurnosnih prijetnji i neovlaštenih pristupa unutar lokalne mreže. U sklopu diplomskog rada potrebno je osmisliti i implementirati mrežnu arhitekturu koja će služiti kao središnja pristupna točka za lokalnu mrežu, omogućavajući usmjeravanje i analizu mrežnog prometa. Potrebno je postaviti i konfigurirati odgovarajuća programska rješenja za inspekciju mrežnih sjednica i detekciju upada kako bi se zabilježili komunikacijski obrasci i identificirali potencijalni indikatori kompromitacije. Na temelju prikupljenih podataka, potrebno je implementirati grafičko sučelje koje će vizualno prikazivati aktivne uređaje u mreži, smjerove njihove komunikacije i evidentirane sigurnosne događaje.

Rok za predaju rada: 26. lipnja 2026.

*Zahvaljujem svojoj obitelji i svima ostalima koji su bili samnom do kraja ovog putovanja.*

*Mentoru na vodstvu i savjetima tokom pisanja rada.*

# Sažetak

## Implementacija središnjeg sustava za nadzor i vizualizaciju komunikacijskih obrazaca lokalne mreže

Ante Čavar

Mala poduzeća danas nemaju konkretan plan obrane od napada na svoju unutarnju mrežu, što uvelike ugrožava njihov opstanak i daljnji rad. Ovaj rad nudi mogućnost malim poduzećima i fizičkim osobama da bez velikih financijskih troškova i skupog sklopovlja brane i nadziru svoju lokalnu mrežu kao i ponašanje uređaja u toj mreži. Sustav omogućuje nadzor i evidenciju mrežnog prometa, vizualizaciju toka podataka u mreži i odgovor na najčešće prijetnje. Najzahtjevniji dio, postavljanje sustava, sada se provodi samo jednom te daljnji rad sustava ne predstavlja akumulirajući trošak, što uvelike pomaže onima kojima je potrebno jednostavno, samoodrživo i prenosivo rješenje za sigurnost lokalne mreže. Rješenje je lako izmjenjivo, što znači da se rad može prilagoditi različitim okruženjima i potrebama korisnika te se time izdvaja od već gotovih i poznatih rješenja.

**Ključne riječi:** Lokalna mreža, Zeek, mrežni obrasci, Docker, IDS, InfluxDB, Elasticsearch

# **Abstract**

## **Implementation of a centralized system for monitoring and visualizing local network communication patterns**

Ante Čavar

Small businesses today lack a concrete plan for defending against attacks on their internal network, which significantly threatens their survival and continued operation. This work offers small businesses and private individuals the ability to defend and monitor their local network, as well as the behavior of the devices within it, without large financial costs or expensive hardware. The system enables the monitoring and logging of network traffic, the visualization of data flow within the network, and a response to the most common threats. The most demanding part, setting up the system, is performed only once, so the system's ongoing operation does not constitute an accumulating cost — which greatly benefits those who need a simple, self-sustaining, and portable solution for local-network security. The solution is easily modifiable, meaning it can be adapted to different environments and user needs, thereby setting it apart from existing, well-known solutions.

**Keywords:** Local network, Zeek, network patterns, Docker, IDS, InfluxDB, Elasticsearch

# Sadržaj

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Uvod</b>                                                      | <b>1</b>  |
| <b>2. Teorija praćenja, pohrane i vizualizacije mrežnog prometa</b> | <b>3</b>  |
| 2.1. Mrežna infrastruktura i pristupne točke                        | 3         |
| 2.2. Sustavi za detekciju upada i analizu prometa                   | 4         |
| 2.3. Kontejnerizacija mrežnih servisa                               | 5         |
| 2.4. Pohrana vremenskih nizova i vizualizacija                      | 5         |
| <b>3. Arhitektura središnje pristupne točke</b>                     | <b>8</b>  |
| 3.1. Pregled sustava                                                | 8         |
| 3.2. Usmeravanje, NAT i pristupna točka                             | 9         |
| 3.3. Praćenje i zapisivanje prometa Zeekom                          | 10        |
| 3.4. Tok podataka i pohrana                                         | 11        |
| 3.5. Prepoznavanje i automatska reakcija                            | 12        |
| <b>4. Implementacija sustava u Docker okruženju</b>                 | <b>14</b> |
| 4.1. Postavljanje Docker okruženja                                  | 14        |
| 4.2. Konfiguracija mreže, NAT-a i pristupne točke                   | 16        |
| 4.3. Konfiguracija Zeek senzora                                     | 19        |
| 4.4. Telegraf/InfluxDB i Filebeat/Elasticsearch                     | 21        |
| 4.5. Grafana nadzorne ploče i usluživanje                           | 23        |
| 4.6. Automatska reakcija: auto_reaction.py i nftables               | 25        |
| 4.7. Izazovi pri implementaciji                                     | 27        |
| <b>5. Evaluacija podsustava za pohranu</b>                          | <b>29</b> |
| 5.1. Metodologija usporedbe                                         | 29        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 5.2. Potrošnja prostora na disku . . . . .                                 | 30        |
| 5.3. Potrošnja radne memorije . . . . .                                    | 31        |
| 5.4. Kašnjenje upita . . . . .                                             | 31        |
| 5.5. Rasprava rezultata . . . . .                                          | 32        |
| 5.6. Zaključak usporedbe . . . . .                                         | 33        |
| <b>6. Testiranje detekcije i automatske reakcije . . . . .</b>             | <b>35</b> |
| 6.1. Testno okruženje i metodologija . . . . .                             | 35        |
| 6.2. Provjera toka podataka . . . . .                                      | 36        |
| 6.3. Testiranje detekcije . . . . .                                        | 36        |
| 6.3.1. Skeniranje portova . . . . .                                        | 37        |
| 6.3.2. Bruteforce SSH prijave . . . . .                                    | 38        |
| 6.3.3. Podudaranje s indikatorom kompromitacije . . . . .                  | 38        |
| 6.4. Testiranje automatske reakcije . . . . .                              | 39        |
| 6.4.1. Blokiranje izvora i istek blokade . . . . .                         | 39        |
| 6.4.2. Zaštita vlastite infrastrukture popisom dopuštenih adresa . . . . . | 40        |
| 6.5. Ograničenja uočena testiranjem . . . . .                              | 41        |
| 6.5.1. Lažiranje izvorišne adrese . . . . .                                | 41        |
| 6.5.2. Otpornost na nasilni prekid skripte . . . . .                       | 42        |
| 6.5.3. Izloženost nadzornih servisa . . . . .                              | 42        |
| <b>7. Zaključak i smjernice za daljnji rad . . . . .</b>                   | <b>43</b> |
| <b>Literatura . . . . .</b>                                                | <b>46</b> |
| <b>Skraćenice . . . . .</b>                                                | <b>48</b> |
| <b>Izjava o korištenju umjetne inteligencije . . . . .</b>                 | <b>50</b> |

# 1. Uvod

Velik broj današnjih uređaja povezan je u neku mrežu, a velika većina njih pripada lokalnim mrežama. Lokalne mreže su time postale sveprisutne, no njihova sigurnost nije pratila brzinu razvoja i pristupačnosti tih istih mreža. Tek nakon niza sigurnosnih incidenata počinje rasti svijest poslovnih i privatnih korisnika o potrebi za zaštitom.

Kako bi se incidenti, a posljedično i troškovi za poslovne korisnike, smanjili, na tržištu se pojavilo mnoštvo programskih rješenja [1] [2], pa i zasebnih zanimanja poput analitičara u sigurnosno-operativnom centru (SOC). Takva su rješenja dostupna prije svega onima koji ih mogu platiti. Za neprekidan nadzor većim je organizacijama isplativo zaposliti tim analitičara raspoređenih u smjene. Postoje i automatizirana rješenja niže razine zaštite, no zajednički im je nedostatak cijena odnosno manjak resursa.

Cilj je ovog rada ponuditi programsko rješenje namijenjeno privatnim korisnicima i manjim tvrtkama, kojima visoki troškovi profesionalnih sustava predstavljaju prepreku koju veće organizacije lakše preskoče. U radu su razrađena i ispitana dva moguća rješenja navedenog problema. Kao temeljni alat za prikupljanje podataka korišten je Zeek, dok je analiza prikupljenih podataka provedena nad dvama podsustavima za pohranu - baza InfluxDB uz agenta Telegraf te baza Elasticsearch uz agenta Filebeat. U oba se slučaja prikupljeni podatci vizualiziraju u alatu Grafana.

Ostatak rada organiziran je na sljedeći način. U drugom poglavlju opisuju se odabrane tehnologije - pristupna točka i usmjeravanje na Linuxu, Zeek kao mrežni senzor, baze InfluxDB i Elasticsearch s pripadnim agentima te Grafana kao vizualizacijsko sučelje. U trećem poglavlju iznosi se arhitektura sustava: usmjeravanje prometa kroz pristupnu točku, mjesto i način praćenja Zeekom te tok podataka prema dvama podsustavima za pohranu. U četvrtom poglavlju opisuje se implementacija svih komponenata u Docker okruženju,

konfiguracija mreže i Zeek senzora te skripta za automatsku reakciju. U petom poglavlju uspoređuju se dva podsustava za pohranu prema zauzeću diska, potrošnji radne memorije i kašnjenju upita. Unutar šestog poglavlja funkcionalno se provjeravaju detekcija sumnjivog prometa i automatska reakcija. U posljednjem, sedmom poglavlju, iznose se zaključci i smjernice za daljnji rad.

## 2. Teorija praćenja, pohrane i vizualizacije mrežnog prometa

Sigurnosni nadzor mreže (engl. *Network Security Monitoring - NSM*) predstavlja metodu prikupljanja, analize i eskalacije indikatora koji ukazuju na maliciozne aktivnosti unutar nekog mrežnog okruženja. Tradicionalni mehanizmi zaštite poput vatrozida i antivirusa se fokusiraju na prevenciju napada izgradnjom barijera tj. statičke obrane. Usprkos tome, kako ističe Bejtlich [3], svaka obrana s vremenom zakaže zato što napadači stalno traže nove načine za zaobilazanje tih obrana - vječna igra mačke i miša. NSM se zato ne oslanja isključivo na spriječavanje i statičku obranu nego i na dubinsku vidljivost mrežnog prometa koja omogućava pravovremenu detekciju i reakciju odnosno aktivnu obranu za incidente.

Kako bi se uspostavio funkcionalan NSM sustav, potrebno je integrirati nekoliko različitih klasa alata: od mrežnih senzora za ekstrakciju metapodataka, preko optimiziranih baza podataka za pohranu vremenskih nizova, do grafičkih sučelja za vizualizaciju anomalija. U nastavku ovog poglavlja bit će opisane tehnologije koje su odabrane kao temelj za izgradnju takvog rješenja.

### 2.1. Mrežna infrastruktura i pristupne točke

AP (engl. *Access Point*) odnosno pristupna točka je uređaj koji se ponaša kao most (engl. *bridge*) između bežičnih uređaja i usmjernika te time omogućava uređajima poput laptopa, mobitela, IoT senzora da se povežu sa žičanom lokalnom mrežom. Za razliku od potrošačkih bežičnih usmjernika, koji u jednom kućištu objedinjuju usmjeravanje, filtriranje prometa i bežično povezivanje, pristupna se točka ograničava isključivo na bežično povezivanje.

Alat *hostapd* omogućuje da se odabrano bežično sučelje na Linuxu pretvori u pristupnu točku [4]. Među njegovim mogućnostima ističu se:

- upravljanje pristupnom točkom prema normama serije IEEE 802.11;
- podrška autentifikacijskim protokolima poput IEEE 802.1X, WPA, WPA2, WPA3;
- rad u ulozi RADIUS poslužitelja ili klijenta, što je značajno korisnicima kojima je potrebna centralizirana autentikacija.

Uređaji povezani na pristupnu točku trebaju dobiti mrežne parametre i mogućnost razlučivanja imena domena. Te usluge u sustavima s ograničenim resursima često pruža alat dnsmasq [5]. Potrebno je naglasiti da dnsmasq ne obavlja usmjeravanje ili filtriranje mrežnog prometa, što su primarni zadatci usmjernika odnosno vatrozida, već objedinjuje funkcionalnosti DNS prosljeđivača (engl. *DNS forwarder*) i DHCP poslužitelja. Alat prihvaća lokalne DNS upite te ih prosljeđuje vanjskim DNS poslužiteljima kako bi se imena (domene) prevela u odgovarajuće IP adrese.

## 2.2. Sustavi za detekciju upada i analizu prometa

Zeek je platforma otvorenog koda [6] namijenjena dubinskoj analizi mrežnog prometa. Za razliku od pristupa koji bilježe sirove pakete (Wireshark), Zeek za svaku promatranu vezu generira strukturirani transakcijski zapis (engl. *transaction log*) s metapodacima poput trajanja veze, broja prenesenih okteta i korištenog protokola. Format izlaznih zapisa moguće je prilagoditi, a analiza se provodi neovisno o veličini mreže.

Zeek posjeduje vlastiti skriptni jezik kojim se definiraju pravila detekcije i određuje koji se metapodaci izdvajaju iz prometa. Time se logika detekcije prilagođava specifičnostima nadzirane mreže bez izmjene jezgre sustava.

Zeek razvrstava zapise prema protokolu i vrsti zabilježene aktivnosti (`conn.log` za sve zamijećene veze, `dhcp.log` za dodjelu lokalnih IP adresa uređajima, `http.log` za praćenje HTTP zahtjeva, `files.log` za praćenja prijenosa datoteka, `dns.log` za praćenja prevođenja simboličkih imena u adrese i drugi). Uz zapise vezane uz pojedini protokol, okvir za obavijesti (engl. *notice framework*) bilježi izvedene sigurnosne događaje u datoteku `notice.log`. Detekcijska se logika i popis prijetnji zadaju kroz `local.zeek` datoteku koja sadrži pravila detekcije napisana Zeekovim skriptnim jezikom, dok `intel.dat` datoteka sadrži popis indikatora kompromitacije koji se učitava neovisno o toj logici.

Alternativni alati poput Suricate [7] primarno su usmjereni na detekciju temeljenu na potpisima (engl. *signature-based detection*). Sveobuhvatna rješenja poput Security Oniona zahitiavaju znatno više resursa, najmanje 12 GB radne memorije za minimalnu instalaciju [2, 8], zbog čega nisu prikladna za potrošačko sklopovlje. Prednost je Zeeka u olakšanoj ekstrakciji metapodataka i svestranom skriptnom jeziku koji omogućuje izradu vlastite detekcijske logike uz malen memorijski otisak.

## 2.3. Kontejnerizacija mrežnih servisa

Docker je platforma za razvoj, dijeljenje i pokretanje aplikacija. Docker omogućava odvajanje aplikacija od infrastrukture [9].

Kontejneri su izolirana okruženja koja se izvode na računalu domaćinu, odvojena od ostalih procesa korištenjem prostorâ imena (engl. *namespaces*) i kontrolnih grupa (engl. *control groups*, *cgroups*). Svaki je kontejner instanca slike (engl. *image*) koju je moguće izraditi, pokrenuti, zaustaviti, premjestiti ili izbrisati Docker alatima. Slike su prenosive, pa se kontejneri mogu izvoditi lokalno, u virtualnim strojevima i u oblaku.

Zbog izolacije prostorima imena i kontrolnim grupama kontejner ima malen utjecaj na domaćina, a za njegovo pokretanje nije potrebno instalirati dodatne ovisnosti na domaćinu. Svaki kontejner je izolirano okruženje te može komunicirati sa ostalim kontejnerima tek ako mu se to eksplicitno dozvoli.

Za orkestraciju više kontejnera koji zajedno čine jednu aplikaciju Docker nudi alat Docker Compose. Konfiguracija cijelog skupa servisa definira se u jednoj *docker-compose.yml* datoteci koja opisuje svaki servis, njegove ovisnosti, volumene i mrežne postavke. Pokretanje cijelog sustava svodi se na jednu naredbu `docker compose up -d`, što znatno pojednostavljuje upogonjavanje i reproducibilnost okruženja te je glavni razlog zašto je upravo Docker korišten u ovom radu.

## 2.4. Pohrana vremenskih nizova i vizualizacija

InfluxDB je baza podataka temeljena na modelu vremenskih nizova (engl. *time-series*), optimizirana za pohranu, indeksiranje i upite nad podacima kod kojih je vrijeme glavna organizacijska dimenzija [10]. Glavne karakteristike InfluxDB su:

- indeksiranje temeljeno na vremenu;
- pohrana u kojoj je moguće samo dodavanje (engl. *append*);
- optimizirana za velik broj sekvencijalnih upisa;
- uklanjanje podataka provodi se politikama zadržavanja (engl. *retention policy*), a ne pojedinačnim izmjenama;
- kompresija i smanjenje uzorkovanja (engl. *downsampling*) radi manjeg zauzeća prostora;
- ugrađene agregacijske funkcije nad vremenskim intervalima (pomični prosjeci, percentili, sažetci).

U ovom je radu korišten Flux, funkcijski skriptni jezik otvorenog koda za upite, analizu i obradu podataka vremenskih nizova te izvorni upitni jezik za InfluxDB 2.x [11, 12]. Za razliku od InfluxQL-a, koji je sintaksno blizak SQL-u, Flux se temelji na kompoziciji operacija povezanih u cjevovod te uz InfluxDB podržava i druge izvore podataka, primjerice PostgreSQL i BigQuery, kao i formate CSV i JSON.

Primjer Flux upita prikazan je u ispisu 2.1 [13].

```
import "influxdata/influxdb/v1"

option v = {bucket: "my-bucket", org: "my-org"}

from(bucket: v.bucket)
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "cpu")
  |> mean()
```

**Kod 2.1.** Primjer Flux upita

Upit 2.1 definira spremnik informacija (engl. *bucket*) te iz njega redom povlači informacije na sljedeći način: svi zapisi iz prethodnih sat vremena (`start: -1h`), koji u sebi sadrže informaciju o CPU-u (`filter(fn: (r) => r._measurement == "cpu")`) te potom iz tih informacija izvlači srednja vrijednost (`mean()`). Drugim riječima ovaj upit će prikupiti informacije o CPU-u u zadnjih sat vremena te će prikazati njihovu srednju vrijednost.

Telegraf je agent otvorenog koda za prikupljanje podataka koji razvija InfluxData, a koji prikuplja, obrađuje i agregira metrike, zapise i druge podatke iz različitih izvora. Telegraf

služi kao primarni sloj za unos podataka u InfluxDB, bazu podataka vremenskih serija, koristeći arhitekturu temeljenu na dodacima s preko 300 ulaznih i izlaznih dodataka za prikupljanje podataka iz sustava, senzora i aplikacija te njihov izravan zapis u InfluxDB [14] [15]. Neke od karakteristika Telegrafa su:

- arhitektura temeljena na dodacima;
- napisan u jeziku Go;
- pretprocesiranje podataka;

Uz InfluxDB i Telegraf, kao alternativa ispitan je i Elasticsearch s Filebeat agentom za prikupljanje podataka.

Elasticsearch je distribuirani sustav za pretraživanje i analizu podataka otvorenog koda temeljen na Apache Lucene biblioteci [16]. Za razliku od InfluxDB-a koji optimizira pohranu vremenskih serija, Elasticsearch tretira svaki zapis kao JSON dokument koji se pohranjuje kroz invertirani indeks (engl. *inverted index*) - struktura podataka koja omogućava brzu pretragu po sadržaju polja. Ta arhitektura čini Elasticsearch pogodnim za pretraživanje punim tekstom i korelaciju podataka iz heterogenih izvora, no dolazi s većim memorijskim i prostornim otiskom u usporedbi s InfluxDB-om što će biti detaljnije prikazano u poglavlju evaluacije.

Kao što Telegraf unosi podatke u InfluxDB, Filebeat preuzima analognu ulogu u ekosustavu Elasticsearcha [17]. Filebeat je lagan agent koji prati zadane datoteke zapisnika i nove zapise šalje Elasticsearchu, koji ih potom indeksira. U ovom radu Filebeat čita Zeekove JSON zapise iz direktorija dijeljenog s ostalim servisima i prosljeđuje ih Elasticsearchu bez transformacije.

Grafana je platforma otvorenog koda za vizualizaciju i analizu podataka koja podržava više od pedeset izvora podataka, uključujući InfluxDB i Elasticsearch [18]. Kontrolne ploče se konfiguriraju kroz grafičko sučelje i mogu se izvesti kao JSON datoteke te pohraniti u repozitorij, što omogućava automatsko učitavanje pri pokretanju sustava (engl. *provisioning*). U ovom radu Grafana služi kao jedinstveno vizualizacijsko sučelje za oba ispitana pozadinska rješenja.

### 3. Arhitektura središnje pristupne točke

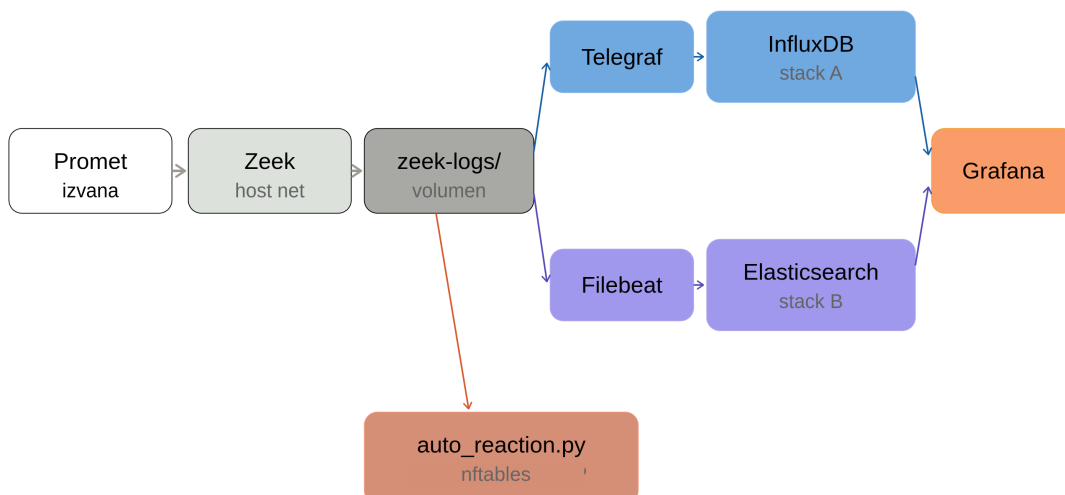
U ovom se poglavlju opisuje arhitektura sustava. Opisuje se način na koji je lokalna mreža usmjerena kroz pristupnu točku, gdje se i kako prikuplja mrežni promet te kako se prikupljeni podatci pohranjuju, vizualiziraju te u konačnici kako se s tim podacima rukuje kako bi se postigla automatska detekcija. Naglasak je na odlukama o dizajnu i njihovim opravdanjima, dok se konkretna implementacija obrađuje kasnije.

#### 3.1. Pregled sustava

Sustav je zamišljen da pretvori računalo s odgovarajućim sklopovljem u jedinstvenu točku kroz koju prolazi sav promet lokalne mreže, čime se ostvaruje potpuna vidljivost komunikacije između lokalnih uređaja i vanjskih odredišta. Računalo istovremeno obavlja funkciju usmjernika (engl. *router*), pristupne točke, mrežnog senzora i vatrozida. Ovakvom tipu posla najviše odgovara računalo koje na sebi ima instaliran Linux, te iako su mogući i drugi načini oni se kroz rad ne istražuju.

Slika 3.1. prikazuje tok podataka kroz sustav. Sav mrežni promet prolazi kroz Zeek koji ga analizira i pohranjuje u obliku strukturiranih JSON zapisa u direktorij dijeljen s ostalim servisima. Tom direktoriju pristupaju dva različita podsustava za pohranu, no ne istovremeno. Prvi podsustav čini vremenski orijentirana baza InfluxDB, u koju podatke unosi agent Telegraf. Drugi čini dokumentno orijentirana baza Elasticsearch, u koju podatke unosi agent Filebeat. Oba podsustava kao zajedničko vizualizacijsko sučelje koriste Grafanu. Postojanje dvaju podsustava ne predstavlja produkcijsku konfiguraciju, nego služi njihovoj usporedbi provedenoj u poglavlju o evaluaciji. U produkcijskom bi se okruženju odabrao jedan od njih, ovisno o potrebama korisnika i mogućnostima dostupnog sklopovlja.

Svi navedeni servisi izvode se u Docker kontejnerima, čime se postiže ponovljivost i pre-

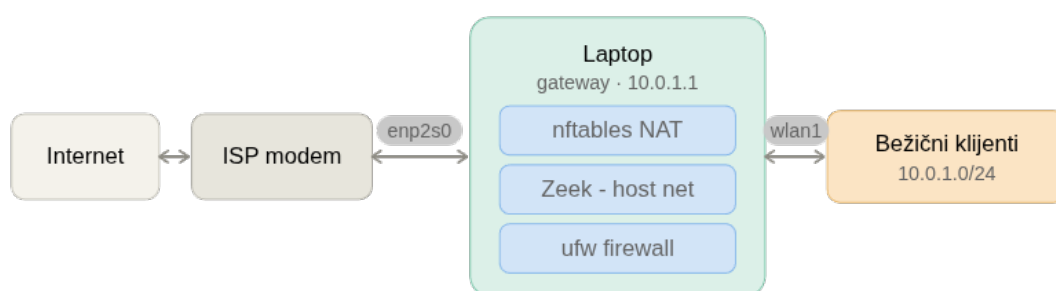


**Slika 3.1.** Prikaz toka podataka kroz sustav

nosivost okruženja te jednostavnost pri postavljanju [9]. Jedina je iznimka skripta za automatsku reakciju, koja se izvodi izravno na računalu domaćinu jer mijenja njegova vatrozidna pravila, o čemu je više riječi u kasnijem odjeljku.

## 3.2. Usmjeravanje, NAT i pristupna točka

Mrežna topologija sustava prikazana je na slici 3.2.



**Slika 3.2.** Mrežna topologija

Računalo koje objedinjuje uloge prikazane na slici 3.2. mora posjedovati barem dva mrežna sučelja. Jedno je žično sučelje povezano prema pružatelju internetskih usluga (engl. *Internet Service Provider, ISP*) i služi kao izlaz prema internetu. Drugo je bežično sučelje u ulogu pristupne točke, na koje se povezuju ostali uređaji lokalne mreže. Bežično se po-

vezivanje moglo iskoristiti i za izlaz prema internetu, no žična je veza u pravilu stabilnija. Konkretni nazivi sučelja i pripadna konfiguracija iznose se u poglavlju o implementaciji.

Lokalni uređaji dobivaju adrese iz privatnog adresnog raspona 10.0.1.0/24, u kojem računalo s ulogom usmjernika ima adresu 10.0.1.1, u skladu s rasponima rezerviranim za privatne mreže [19]. Bežično sučelje u ulogu pristupne točke postavlja alat `hostapd`, dok mrežne parametre klijentima dodjeljuje alat `dnsmasq` u ulozi DHCP poslužitelja.

Budući da lokalni uređaji vanjskim mrežama pristupaju isključivo preko ovog računala, sav promet između njih i vanjskih odredišta nužno prolazi kroz jednu točku, čime se ostvaruje potpuna vidljivost prometa nužna za analizu komunikacijskih obrazaca. Kako lokalni uređaji dijele jednu javnu adresu domaćina, na izlaznom se sučelju provodi prevođenje mrežnih adresa (engl. *Network Address Translation*, NAT).

Upravljanje vatrozidom na računalu temelji se na dva alata, `nftables` i `ufw`. Oba djeluju nad istom pozadinskom implementacijom. Na suvremenim se distribucijama i `ufw` izvodi kroz `nftables`, a starije `iptables` naredbe prevode se posredstvom sloja `iptables-nft`. `nftables` je zapravo jedini pozadinski mehanizam koji se koristi, dok se naredbe ostalih alata prevode kroz `nft`.

Vatrozid `ufw` (engl. *Uncomplicated Firewall*) provodi pravila filtriranja izravno na domaćinu. Zadana je politika odbacivanje svog prometa, pa se rad pristupne točke mora izrijeком dopustiti minimalnim skupom pravila. Posljedica je da domaćin neželjeni dolazni internetski promet odbacuje već na temelju zadane politike, neovisno o ostalim slojevima. Konkretna se pravila iznose u poglavlju o implementaciji.

### 3.3. Praćenje i zapisivanje prometa Zeekom

Nadzorni sloj čini Zeek, kojem je omogućen pristup bežičnom sučelju radi praćenja prometa. Za razliku od ostalih servisa, Zeek se ne izvodi u izoliranoj kontejnerskoj mreži, nego koristi mrežu domaćina (engl. *host network mode*), jer mora izravno pristupiti fizičkom sučelju radi osluškivanja prometa. Iako mu dodijeljene ovlasti tehnički dopuštaju i slanje paketa, Zeek po svojoj namjeni promet isključivo čita i ne generira ga, pa je njegova uloga u praćenju pasivna. Pojednosti o ovlastima i načinu izvođenja iznose se u poglavlju o implementaciji.

Nadalje, odabir bežičnog sučelja kao mjesta praćenja nije proizvoljan. Praćenjem prije prevođenja adresa izvorišne i odredišne adrese ostaju nepromijenjene, čime se zadržava atribucija prometa po pojedinom uređaju, što je nužno za smislenu analizu. Nije isto ako stolno računalo ili pametni hladnjak pokreću SSH i FTP veze.

Zeek prepoznaje i prati mnoštvo mrežnih protokola te o njihovoj aktivnosti vodi strukturane zapise koje razvrstava u zasebne datoteke prema vrsti zabilježene aktivnosti i pohranjuje te zapise u direktorij `zeek-logs`. Među njima su, primjerice, `conn.log`, `dns.log`, `dhcp.log`, `http.log`, `ssh.log`, `files.log`, `quic.log` i `notice.log`. Većina je datoteka vezana uz pojedini protokol, dok pojedini zapisi bilježe izvedene događaje neovisno o protokolu. Najvažnija takva datoteka je `notice.log`, u koju Zeek upisuje upozorenja na temelju kojih se pokreće automatska reakcija opisana kasnije u ovom poglavlju. Sljedeći važan zapis je `conn.log` u kojem se nalazi zapis preostalog prometa poput TCP, UDP, ICMP. Kratki opis ostalih zapisa nalazi se u tablici 3.1.

| Naziv datoteke         | Opis                                                                                                  |
|------------------------|-------------------------------------------------------------------------------------------------------|
| <code>dns.log</code>   | zapisuje sve DNS upite                                                                                |
| <code>dhcp.log</code>  | zapisuje klijente te informacije o njihovim IP adresama, MAC-ovima, nazivima                          |
| <code>http.log</code>  | zapisuje sve HTTP zahtjeve zato što su nešifrirani i bogati metapodacima za razliku od HTTPS zahtjeva |
| <code>ssh.log</code>   | zapisuje sve SSH veze                                                                                 |
| <code>files.log</code> | sve datoteke koje su prenesene kroz nadzirano sučelje                                                 |
| <code>quic.log</code>  | zapis o brzim UDP vezama (engl. <i>Quick UDP Internet Connections</i> )                               |

Tablica 3.1. Nazivi i opisi datoteka u `zeek-logs`.

Zapisi se generiraju u JSON formatu, čime se omogućuje njihovo izravno strojno čitanje. Time direktorij `zeek-logs` postaje jedinstvena predajna točka prema podsustavima za pohranu, opisanim u nastavku.

## 3.4. Tok podataka i pohrana

Zeekovi zapisi iz direktorija `zeek-logs` su ulazna točka za pohranu, kako je prikazano na slici 3.1. Taj je direktorij u Zeekovom kontejneru montiran s pravom pisanja, a u agente za prikupljanje isključivo s pravom čitanja. Time se prikupljanje odvaja od pohrane.

Nad istim se zapisima ispituju dva podsustava za pohranu. Prvi čini agent Telegraf, koji Zeekove JSON zapise upisuje u vremenski orijentiranu bazu InfluxDB. Drugi čini agent Fi-

lebeat, koji iste zapise prosljeđuje u dokumentno orijentiranu bazu Elasticsearch. Uloge i svojstva pojedinih alata detaljnije su opisani u prethodnom poglavlju o teorijskim osnovama.

Ta dva podsustava nisu predviđena za istovremeni rad. Definirani su u zasebnim konfiguracijama, od kojih svaka sadrži vlastiti senzor i vlastitu instancu Grafane, pa se u danom trenutku izvodi isključivo jedan. Razdvajanje ima dva cilja. Prvo, izolacija sprječava da trošenje resursa jednog podsustava iskrivi mjerenja drugoga, što je preduvjet poštene usporedbe provedene u poglavlju o evaluaciji. Drugo, budući da obje konfiguracije kao ulaz koriste isti skup Zeekovih zapisa istoga formata, razlike u rezultatima moguće je pripisati isključivo pozadinskoj bazi, a ne ulaznim podacima.

### 3.5. Prepoznavanje i automatska reakcija

Prepoznavanje i automatska reakcija odvijaju se u dva koraka. Najprije Zeek prepoznaje kada je neka veza sumnjiva. Podrazumijevano Zeek ne prepoznaje maliciozne veze, no dolazi s većim brojem gotovih detektora i okvira koje je moguće uključiti i prilagoditi. Uz njih se detekcijska logika proširuje i vlastitim pravilima opisanima Zeekovim skriptnim jezikom, kao i indikatorima kompromitacije (engl. *Indicators of Compromise, IoC*) koji se zadaju kao popis adresa neovisan o samoj logici. Kada Zeek prepozna takav događaj, upisuje upozorenje u `notice.log`.

Na to se nadovezuje drugi korak, skripta za automatsku reakciju `auto_reaction.py`. Ona prati novoupisana upozorenja i na temelju njih blokira izvorišne adrese. Skripta se izvodi s administratorskim ovlastima jer mijenja vatrozidna pravila računala domaćina.

Blokiranje je izvedeno preko skupa (engl. *set*) koji sadrži blokirane adrese, a zasebni lanci vatrozida odbacuju sav promet čija se izvorišna adresa nalazi u tom skupu. Ti su lanci postavljeni na viši prioritet od zadanih `ufw` lanaca, pa odbacivanje stupa na snagu prije nego što ostala pravila propuste vezu. Adrese usmjernika i lokalnog domaćina izuzete su popisom dopuštenih adresa (engl. *whitelist*) kako bi se spriječilo blokiranje vlastite infrastrukture.

Nadalje, potrebno je istaknuti i ograničenje autorovog rješenja, odnosno reakcija se temelji na izvorišnoj adresi iz zapisa upozorenja. U slučaju lažiranja izvorišne adrese (engl. *spoofing*) skripta blokira adresu navedenu u zapisu, koja ne mora odgovarati stvarnom napa-

daču. To znači da napadač može lažirati adresu legitimnog korisnika odnosno uređaja. Kako bi se to izbjeglo predlaže se stalno ažuriranje popisa dopuštenih adresa uređajima kojima se vjeruje kao i dodjela statičkih adresa uređajima od povjerenja.

Svaki element prethodno navedenog skupa nosi vremensko ograničenje (engl. *timeout*) kojim upravlja jezgra operacijskog sustava, pa se pravilo uklanja automatski nakon isteka, bez ikakve intervencije korisnika ili skripte. Time se izbjegava slabost izvedbe zasnovane na izravnim pravilima. Ukratko to znači da nasilni prekid skripte (signalom SIGKILL, nedostatkom memorije ili gubitkom napajanja) ne ostavlja zaostala pravila jer ona ionako isteknu sama.

## 4. Implementacija sustava u Docker okruženju

U ovom se poglavlju opisuje konkretna izvedba sustava čija je arhitektura iznesena u prethodnom poglavlju. Redom se prikazuju postavljanje Docker okruženja i način na koji se servisi pokreću jednom skriptom, konfiguracija mreže s pristupnom točkom, prevođenjem adresa i pravilima vatrozida te postavljanje Zeek senzora s vlastitim detektorima. Nakon toga opisuju se dva podsustava za pohranu, Telegraf s InfluxDB-om i Filebeat s Elasticsearchom, te njihova zajednička vizualizacija u Grafani. Poglavlje završava skriptom za automatsku reakciju i pregledom najznačajnijih problema na koje se nailazilo tijekom implementacije.

### 4.1. Postavljanje Docker okruženja

Svi servisi sustava, osim skripte za automatsku reakciju, izvode se u Docker kontejnerima [9]. Kreirane su dvije `docker-compose` datoteke, jedna za svaki od dvaju ispitana podsustava. `docker-compose.yml` opisuje podsustav s bazom InfluxDB i agentom Telegraf, dok `docker-compose-elk.yml` opisuje podustav s bazom Elasticsearch i agentom Filebeat. Obje konfiguracije dijele isti Zeek senzor i istu inačicu Grafane. Budući da oba podsustava koriste kontejner istog imena (`zeek-ids`) i isti port (3000 za Grafanu), istovremeno pokretanje nije moguće, odnosno u bilo kojem danom trenutku se može izvoditi samo jedan podsustav, što je ujedno i preduvjet za poštenu usporedbu u kasnijem poglavlju evaluacije.

Osjetljivi parametri poput korisničkih imena, lozinki, pristupnih tokena te naziva organizacije i spremnika (engl. *bucket*) - izdvojeni su iz `docker-compose` datoteka u zasebnu `.env` datoteku okruženja, čime se osjetljivi i okruženju specifični parametri drže odvojeno od konfiguracije, što je standardna praksa. Struktura datoteke `.env` prikazana je u ispisu 4.1

```
INFLUXDB_USER=admin
INFLUXDB_PASSWORD=<lozinka>
INFLUXDB_ORG=dipl
INFLUXDB_BUCKET=zeek-logs
INFLUXDB_TOKEN=<token>

GRAFANA_USER=admin
GRAFANA_PASSWORD=<lozinka>
```

**Kod 4.1.** Struktura .env datoteke (lozinke i pristupni tokeni skriveni)

Pri prvom pokretanju InfluxDB se inicijalizira automatski putem varijabli DOCKER\_INFLUXDB\_INIT\_\* u setup načinu rada, prikazanog u ispisu 4.2, što omogućuje da se bez ručne intervencije stvaraju korisnik, organizacija, spremnik i administratorski token. Postupak se izvodi samo pri prvom pokretanju, dok se pri kasnijima postojeći podaci zadržavaju. Time se izbjegava ručno postavljanje baze prilikom prvog pokretanja, što je veoma bitno za ponovljivost i održivost okruženja.

```
environment:
  - DOCKER_INFLUXDB_INIT_MODE=setup
  - DOCKER_INFLUXDB_INIT_USERNAME=${INFLUXDB_USER}
  - DOCKER_INFLUXDB_INIT_PASSWORD=${INFLUXDB_PASSWORD}
  - DOCKER_INFLUXDB_INIT_ORG=${INFLUXDB_ORG}
  - DOCKER_INFLUXDB_INIT_BUCKET=${INFLUXDB_BUCKET}
  - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=${INFLUXDB_TOKEN}
```

**Kod 4.2.** Automatska inicijalizacija InfluxDB okruženja iz docker-compose.yml

Isti se token prosljeđuje Grafani kroz varijablu okoline kako bi se izvor podataka konfigurirao automatski pri pokretanju (engl. *provisioning*), bez ručnog unosa pristupnih podataka u sučelju kao što je vidljivo u ispisu 4.3

```
environment:
  - GF_SECURITY_ADMIN_USER=${GRAFANA_USER}
  - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
  - GF_USERS_ALLOW_SIGN_UP=false
  # Prosljeđuje token u provisioning YAML putem env varijable
  - INFLUXDB_TOKEN=${INFLUXDB_TOKEN}
  - INFLUXDB_ORG=${INFLUXDB_ORG}
  - INFLUXDB_BUCKET=${INFLUXDB_BUCKET}
```

**Kod 4.3.** Automatska inicijalizacija Grafane okruženja iz docker-compose.yml

Cjelokupno pokretanje, zaustavljanje i nadzor okruženja centralizira skripta `dipl.sh`, koja povezuje postavljanje pristupne točke i pokretanje kontejnera u jednu cjelinu. Postavljanje se provodi korak po korak. Ako pokretanje kontejnera ne uspije, prethodno postavljena

mrežna konfiguracija automatski se poništava, što je vidljivo u ispisu 4.4, čime se izbjegava ostavljanje sustava u polupostavljenom stanju u kojem je pristupna točka aktivna, a nadzor nije.

```
if [ "$1" == "start" ]; then
echo -e "${YELLOW}[1/2] _Postavljanje _AP _okoline...${NC}"
bash "$NET_SCRIPT" set
if [ $? -ne 0 ]; then
echo -e "${RED}Greska _pri _postavljanju _AP -a. _Prekidam.${NC}"
exit 1
fi
echo -e "${GREEN}AP _okolina _postavljena.${NC}"
echo -e "${YELLOW}[2/2] _Pokretanje _Docker _servisa...${NC}"
docker compose -f "$SCRIPT_DIR/docker-compose.yml" up -d
if [ $? -ne 0 ]; then
echo -e "${RED}Greska _pri _pokretanju _Docker _servisa. _
Resetiram _AP...${NC}"
bash "$NET_SCRIPT" reset
exit 1
fi
fi
```

**Kod 4.4.** Atomarno pokretanje u `dipl.sh` (naredba `start`)

Naredbe `stop`, `restart` i `status` upravljaju gašenjem, ponovnim pokretanjem i prikazom stanja servisa na sljedeći način: `sudo ./dipl.sh <naredba>`. U ispisu su `$NET_SCRIPT` putanja do mrežne skripte, a `${NC}`, `${RED}` i slično oznake boja terminala radi preglednijeg ispisa.

## 4.2. Konfiguracija mreže, NAT-a i pristupne točke

Potrebna mrežna konfiguracija postavlja se skriptom `net_setReset_rules.sh`, koja prima argument `set` ili `reset`. Računalo raspolaže dvama sučeljima - žičanim `enp2s0` prema pružatelju internetskih usluga i bežičnim `wlan1` u ulozu pristupne točke s adresom `10.0.1.1`. Bežično sučelje u ulozu pristupne točke ostvareno je USB bežičnim mrežnim prilagodnikom TP-Link TL-WN722N inačica v1.

Za rad u načinu pristupne točke prilagodnik mora zadovoljiti dva uvjeta. Njegov *chipset* mora podržavati način rada pristupne točke (engl. *master/AP mode*), a pripadni upravljački program mora biti dostupan kroz `n180211` sučelje jezgre kojim `hostapd` upravlja prilagodnikom. Poželjno je i da je upravljački program dio jezgre čime se izbjegava oslanjanje na vanjske, neslužbene upravljačke programe.

Inačica v1 zadovoljava sve navedeno. Temelji se na *chipsetu* Atheros AR9271 s upravljačkim programom *ath9k\_htc*, koji je dio jezgre i izvorno podržava način pristupne točke [20]. Kasnije inačice (v2 i v3) koriste *chipset* Realtek RTL8188EUS, za koji podrška za način pristupne točke nije uključena u standardnu jezgru, već zahtijeva ručno postavljanje neslužbenih upravljačkih programa. Stoga je upravo inačica v1 prikladna za ovu primjenu bez dodatnih zahvata.

Sučelje wlan1 najprije se izuzima iz nadzora servisa NetworkManager, nakon čega mu se ručno dodjeljuje adresa i omogućuje prosljeđivanje paketa kao što je vidljivo u ispisu 4.5 Izuzimanje je nužno jer bi NetworkManager inače pokušao samostalno upravljati sučeljem i ulazio u sukob s *hostapd*-om koji isto sučelje postavlja u način rada pristupne točke.

```
nmcli device set wlan1 managed no
ip link set wlan1 down
ip link set wlan1 up
ip addr add 10.0.1.1/24 dev wlan1
sysctl -w net.ipv4.ip_forward=1
```

**Kod 4.5.** Izuzimanje sučelja i omogućavanje usmjeravanja

Konfiguracija vatrozida i prevođenja adresa smješteni su u zasebnu *nftables* tablicu diplomski, što omogućuje uklanjanje postavki jednom naredbom bez utjecaja na ostatak sustava. Tablica sadrži pravilo maskiranja (engl. *NAT*) odlaznog prometa pod mreže 10.0.1.0/24 na sučelju *enp2s0*.

Uz pravilo maskiranja, ista tablica unaprijed priprema infrastrukturu za automatsku reakciju pomoću skupa *denylist* s vremenskim ograničenjem trajanja elemenata te dva lanca koja odbacuju promet izvorišnih adresa iz tog skupa na *forward* i *input* kukama (engl. *hooks*) 4.6 Lanci su postavljeni na prioritet -10. U *nftables* niži broj prioriteta znači ranije izvođenje, pa se ova pravila primjenjuju prije zadanih *ufw* lanaca, čime odbacivanje stupa na snagu prije nego što ostala pravila propuste vezu. Struktura blokade definira se ovdje, jednom, dok skripta za reakciju opisana u kasnijoj sekciji samo dodaje adrese u postojeći skup.

```
nft add set ip diplomski denylist { type ipv4_addr ; flags
    timeout ; timeout 30s ; }

nft add chain ip diplomski drop_fwd { type filter hook forward
    priority -10 ; }
nft add rule ip diplomski drop_fwd ip saddr @denylist drop
nft add chain ip diplomski drop_in { type filter hook input
    priority -10 ; }
```

```
nft add rule ip diplomski drop_in ip saddr @denylist drop
```

**Kod 4.6.** denylist i lanci za odbacivanje prometa

Originalna politika vatrozida ufw odbacuje sav promet, pa se rad pristupne točke iznimkom dopušta najmanjim potrebnim skupom pravila 4.7 odnosno DHCP promet na portovima 67 i 68 te usmjeravanje s wlan1 na enp2s0. Posljedica te je politike da će se dolazni neželjeni internetski promet odbaciti neovisno o ostalim slojevima.

```
ufw allow in on wlan1 to any port 67 proto udp
ufw allow in on wlan1 to any port 68 proto udp
ufw route allow in on wlan1 out on enp2s0
```

**Kod 4.7.** Dopuštenja ufw za rad pristupne točke

Bežična pristupna točka realizirana je alatom hostapd [4] na sučelju wlan1, uz zaštitu WPA2-PSK sa CCMP protokolom, u ispisu 4.8 Ključna je postavka ap\_isolate=0 kojom se onemogućuje izolacija klijenata. Pristupne točke uobičajeno izoliraju klijente kako međusobno ne bi komunicirali, no tada njihov međusobni promet ne bi prolazio kroz točku nadzora; isključivanjem izolacije Zeek dobiva vidljivost i nad istok-zapad prometom (promet između uređaja) unutar lokalne mreže.

```
interface=wlan1
ssid=Dipl_test_FORBIDDEN
hw_mode=g
channel=6
wpa=2
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
wpa_passphrase=<lozinka>
ap_isolate=0
```

**Kod 4.8.** Bitne postavke hostapd konfiguracije

Mrežne parametre postavlja dnsmasq [5] prema ispisu 4.9 tako da klijentima na sučelju wlan1 dodjeljuje adrese iz raspona 10.0.1.10–10.0.1.100, zadaje usmjernik 10.0.1.1 te adrese vanjskih DNS poslužitelja 8.8.8.8 i 1.1.1.1. U ovoj konfiguraciji dnsmasq djeluje isključivo kao DHCP poslužitelj, dok se prevođenje adresa povjerava izravno vanjskim poslužiteljima, bez lokalnog prosljeđivanja. Time se ne gubi nadzor nad DNS prometom, unatoč tome da su odredišni DNS poslužitelji izvan lokalne mreže, upiti svejedno prolaze kroz usmjernik i sučelje wlan1, gdje ih Zeek bilježi i pridodaje pojedinom klijentu odnosno uređaju. Ova vidljivost vrijedi za klasične DNS upite preko UDP/53; klijenti koji koriste šifrirani DNS (DoH ili DoT) zaobilaze je jer Zeek takve upite vidi tek kao TLS odnosno

QUIC promet prema vanjskom poslužitelju. Korišteni adresni prostor 10.0.1.0/24 dio je raspona 10.0.0.0/8 rezerviranog za privatne mreže prema RFC-u 1918 [19], dok dodjela adrese s završetkom .1 usmjerniku slijedi uobičajenu, premda nestandardiziranu, praksu pri postavljanju lokalnih mreža.

```
interface=wlan1
dhcp-range=10.0.1.10,10.0.1.100,255.255.255.0,24h
dhcp-option=option:router,10.0.1.1
dhcp-option=option:dns-server,8.8.8.8,1.1.1.1
```

**Kod 4.9.** Postavke DHCP-a i DNS prosljeđivanja u dnsmasq

Naredbom `sudo ./dipl.sh stop` u glavnom direktoriju cijela se konfiguracija poništava. Gase se `hostapd` i `dnsmasq`, briše se `nftables` tablica `diplomski`, uklanjaju se `ufw` iznimke, vraća se `ip_forward` na 0 te se sučelje `wlan1` vraća pod nadzor `NetworkManager`a.

### 4.3. Konfiguracija Zeek senzora

Zeek se pokreće kao servis u obje `docker-compose` datoteke, prema ispisu 4.10 Za razliku od ostalih servisa koji koriste izoliranu bridge mrežu, Zeek koristi mrežu domaćina (`network_mode: host`), pa mu je fizičko sučelje `wlan1` izravno dostupno bez zasebnog mapiranja; u izoliranoj mreži kontejner ne bi vidio promet domaćinskog sučelja. Zbog toga se njegovom kontejneru dodjeljuju ovlasti `NET_ADMIN` i `NET_RAW`, koje omogućuju otvaranje sirovih spojnih točaka (engl. *raw sockets*) i hvatanje paketa na razini sučelja.

```
zeek:
  image: zeek/zeek:8.0.6
  network_mode: "host"
  cap_add: [ NET_ADMIN, NET_RAW ]
  volumes:
    - ./zeek-logs:/usr/local/zeek/logs
    - ./zeek/local.zeek:/usr/local/zeek/share/zeek/site/local.zeek:ro
    - ./zeek/intel.dat:/usr/local/zeek/share/zeek/site/intel.dat:ro
  command: zeek -C -i wlan1 local
```

**Kod 4.10.** Definicija senzora u `docker-compose.yml`

Servis se pokreće naredbom `zeek -C -i wlan1 local`. Argument `-C` isključuje provjeru kontrolnih zbrojeva (engl. *checksum*) paketa što je nužno jer mrežne kartice često računaju zbrojeve sklopovski, pa ih jezgra Zeeku predaje kao naizgled neispravne. `-i wlan1` određuje sučelje za nadzor, a argument `local` učitava skriptu `site/local.zeek`. Volumeni

razdvajaju pisanje od čitanja tako što je zeek-logs montiran s pravom pisanja kako bi senzor u njega zapisivao, dok su local.zeek i intel.dat montirani isključivo za čitanje što je vidljivo iz dodatka :ro što znači *read-only*.

Skripta local.zeek učitava bazne skenere protokola, okvir za obavijesti i proširenje json-logs pomoću kojega se zapisi pišu u JSON formatu prikladnom za izravno strojno čitanje agenata. Učitavanje se u datoteci local.zeek provodi direktivom @load, kojom se uključuju Zeekovi ugrađeni okviri i skeneri pojedinih protokola. Ispis 4.11 prikazuje učitavanje baznih skenera za promatrane protokole te okvira za obavijesti, čime se određuje koje protokole Zeek raščlanjuje i o kojima vodi zapise.

```
@load base/protocols/conn
@load base/protocols/dns
@load base/protocols/http
@load base/protocols/ssl
@load base/frameworks/notice
```

**Kod 4.11.** Učitavanje ugrađenih dodataka u local.zeek

Bez učitanih dodataka Zeek promatra manji skup protokola, pa je učitavanjem moguće proširiti opseg praćenja prema potrebama nadzirane mreže. Uz Zeekov ugrađeni SSH (engl. *bruteforce*) detektor, u skripti su definirani i vlastite detektori. Ispis 4.12 prikazuje detekciju skeniranja portova.

```
global port_scan_tracker: table[addr, addr] of set[port] &
    create_expire=5min;

event new_connection(c: connection)
{
    local src = c$id$orig_h;
    local dest = c$id$resp_h;
    local dest_p = c$id$resp_p;

    if ([src, dest] !in port_scan_tracker)
        port_scan_tracker[src, dest] = set();
    add port_scan_tracker[src, dest][dest_p];

    if (|port_scan_tracker[src, dest]| >= 50)
        NOTICE([$note=Custom_Port_Scan, $src=src,
            $msg=fmt("Moguci port scan: %s kontaktirao %d
                razlicitih portova na %s",
                    src, |port_scan_tracker[src, dest]|,
                    dest),
            $identifier=cat(src, dest), $suppress_for=30sec
        ]);
}
```

**Kod 4.12.** Detekcija skeniranja portova u local.zeek

Najprije nastaje tablica za praćenje ulaza koja se indeksira s parom [izvorišna adresa, odredišna adresa] a njihova vrijednost je skup portova koji su korišteni u komunikaciji. Ukoliko je unutar pet minuta kontaktirano 50 ili više portova sa istim parom adresa, podiže se obavijest u `notice.log` i taj tip obavijesti se utišava na 30 sekundi kako upozorenja ne bi zagušavala njihov zapisnik. Portovi se brišu iz skupa nakon pet minuta kako bi se očistila memorija. Analognim se pristupom prati i učestalost SSH konekcija prema usmjerniku.

Indikatori kompromitacije (engl. *Indicators of Compromise*) učitavaju se iz datoteke `intel.dat` putem Zeekovog Intel okvira [6]. Adrese se navode u datoteci, a ne u skripti, čime se popis prijetnji može mijenjati bez izmjene detekcijske logike.

Svi se zapisi razvrstavaju prema vrsti aktivnosti (`conn.log`, `dns.log`, `http.log`, `notice.log` i drugi) i pohranjuju u dijeljeni direktorij `zeek-logs`, kojem pristupaju podsustavi za pohranu opisani u nastavku.

## 4.4. Telegraf/InfluxDB i Filebeat/Elasticsearch

Podsustavi za pohranu kao ulaz koriste isti skup Zeekovih zapisa iz dijeljenog direktorija `zeek-logs`, montiranog isključivo za čitanje. Uzimaju se dva zapisa: `conn.log`, koji nosi najveći dio prometa, i `notice.log`, koji sadrži upozorenja. Time se u obje baze unosi istovjetan ulaz, što je preduvjet usporedbe iz poglavlja evaluacije. Razlike u rezultatu ovise jedino o pozadinskoj bazi i njenom agentu, a ne ulaznim podacima.

U podsustavu s InfluxDB-om unos podataka obavlja Telegraf [14]. Ulazni dodatak `inputs.tail` prati `conn.log` i `notice.log`, raščlanjuje svaki redak kao JSON (format `json_v2`) i upisuje ga u InfluxDB putem izlaza `influxdb_v2`, kojem se pristupni token, organizacija i spremnik predaju putem varijabli okoline. Praćenje datoteka koristi metodu `poll` jer se zapisi nalaze na povezanom volumenu, gdje obavješćavanje jezgre o promjenama datoteke (`inotify`) nije pouzdano.

Pri raščlanjivanju razdvajaju se oznake (engl. *tags*) od polja (engl. *fields*). Oznake predstavljaju izvorišna i odredišna adresa, protokol, servis i stanje veze. InfluxDB ih indeksira, pa služe za filtriranje i grupiranje u Grafani. Vrijednosti s velikim brojem različitih pojavnosti, poput portova, količine prenesenih okteta i identifikatora veze `uid`, pohranjuju se kao polja kako se ne bi nepotrebno umnažao broj vremenskih serija. Posebno specifično je rašč-

članjivanje Zeekovih ključeva oblika `id.orig_h` s obzirom da `json_v2` točku tumači kao razdjelnik putanje, a u zapisu je ona dio samog naziva ključa, točka se u putanji mora izuzeti, u ispisu 4.13

```
[[inputs.tail.json_v2.tag]]
  path      = "id\\.orig_h"    # točka je dio naziva ključa, ne
    razdjelnik
  rename    = "src_ip"

[[inputs.tail.json_v2.field]]
  path      = "id\\.resp_p"
  rename    = "dst_port"
  type      = "int"
```

**Kod 4.13.** Izuzimanje točke u nazivu ključa (telegraf.conf)

U podsustavu s Elasticsearchom ulogu agenta preuzima Filebeat [17]. Svaki se Zeekov zapis prati zasebnim ulazom tipa `filestream` koji retke raščlanjuje kao NDJSON, a pripadnu vrstu zapisa označava poljem `zeek_type`. Procesor za vremenske oznake pretvara Zeekov `ts` (Unix vrijeme s decimalama) u standardno polje `@timestamp`. Zapisi se potom šalju u Elasticsearch i pohranjuju u indeks nazvan prema vrsti i datumu, primjerice `zeek-conn-2026.05.29`, vidljivo u ispisu 4.14

```
filebeat.inputs:
  - type: filestream
    id: zeek-conn
    paths: [ /zeek-logs/conn.log ]
    parsers:
      - ndjson: { keys_under_root: true }
      fields: { zeek_type: "conn" }

output.elasticsearch:
  hosts: [ "http://elasticsearch:9200" ]
  index: "zeek-%{[fields.zeek_type]}-%{+yyyy.MM.dd}"
```

**Kod 4.14.** Ulaz i odredišni indeks u filebeat.yml

Ostali tipovi zapisa (`dns.log`, `http.log`, `ssl.log`, `dhcp.log`) u konfiguraciji su pripremljeni, ali namjerno isključeni, kako bi oba podsustava unosila istovjetan skup zapisa. Upravljanje predloškom indeksa i politikom životnog ciklusa indeksa (engl. *Index Lifecycle Management*, *ILM*) isključeno je, pa Elasticsearch mapiranje polja stvara dinamički. Posljedice takvog mapiranja na zauzeće prostora razmatrat će se u poglavlju evaluacije.

## 4.5. Grafana nadzorne ploče i usluživanje

Grafana je za podsustave spremanja zajedničko vizualizacijsko sučelje [18], čime se uklanja utjecaj načina prikaza na usporedbu pozadinskih baza. Izvor podataka ne konfigurira se ručno, nego se učitava automatski pri pokretanju (engl. *provisioning*) iz datoteke opisa

izvora, čime se izbjegava ručni unos pristupnih podataka i osigurava ponovljivost. Za podsustav s InfluxDB-om opisuje se izvor tipa `influxdb` s Flux jezikom upita, kojem se organizacija, spremnik i token predaju varijablama okoline, u ispisu 4.15, a za podsustav s Elasticsearchom opisuje se istovjetan izvor tipa `elasticsearch` nad uzorkom indeksa `zeek-*`. Budući da se podsustavi pokreću zasebno, svaka instanca Grafane učitava isključivo svoj izvor podataka.

```
datasources:
- name: InfluxDB
  type: influxdb
  access: proxy
  url: http://influxdb:8086
  uid: P951FEA4DE68E13C5      # fiksni uid radi podudaranja s
    nadzornom plocom
  jsonData:
    version: Flux
    organization: ${INFLUXDB_ORG}
    defaultBucket: ${INFLUXDB_BUCKET}
  secureJsonData:
    token: ${INFLUXDB_TOKEN}
```

**Kod 4.15.** Automatsko učitavanje InfluxDB izvora podataka

Nadzorna ploča organizirana je u dvije skupine prikaza pregled prometa i sigurnosni pregled. Prikazi se temelje na shemi opisanoj u prethodnom odjeljku, odnosno na zapisima `conn.log` i `notice.log` te pripadnim oznakama i poljima unutar tih zapisa.

Pregled prometa čine prikaz broja veza po minuti (koji broji zapise veza u kliznim jednominutnim prozorima), raspodjela protokola (kružni prikaz dobiven grupiranjem po oznaci proto) te ljestvice najčešćih izvorišnih i odredišnih adresa. Primjer Flux upita, za prikaz broja veza po minuti, dan je u ispisu 4.16 gdje postavljamo period vremena, filter po polju (grupiranje), izbacivanje nepotrebnih stupaca, učestalost uzorkovanja i naziv povratnog upita; oznake se prije zbrajanja uklanjaju kako bi se pojedinačne vremenske serije sažele u jednu vrijednost.

```
from(bucket: "zeek-logs")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "zeek_conn")
  |> filter(fn: (r) => r._field == "uid")
  |> drop(columns: ["src_ip", "dst_ip", "proto", "service", "
    conn_state"])
  |> aggregateWindow(every: 1m, fn: count, createEmpty: false)
  |> yield(name: "connections_per_minute")
```

**Kod 4.16.** Flux upit prikaza broja veza po minuti

Posebno je vrijedan prikaz najčešćih izvorišnih adresa. Budući da Zeek promet hvata na sučelju wlan1 prije prevođenja adresa, izvorišne adrese ostaju stvarne adrese uređaja u lokalnoj mreži (primjerice 10.0.1.94), pa ih je u prikazu moguće i imenovati. Time prikaz izravno potvrđuje vrijednost odluke o mjestu nadzora iz poglavlja arhitekture. Kada bi se hvatanje provodilo nakon prevođenja adresa, svi bi se izvorišni uređaji stopili u jednu adresu usmjernika i atribucija po uređaju bila bi izgubljena.

Sigurnosni pregled čine tablica upozorenja iz `zeek_notice` s tipom i izvorišnom adresom, vremenski prikaz učestalosti upozorenja te tablica sumnjivih veza. Potonja izdvaja veze čije stanje (`conn_state`) odstupa od urednog dovršetka (SF - normalno uspostavljanje i raskid veze) i grupira ih po stanju i izvorišnjoj adresi, čime se ističu obrasci poput velikog broja neuspjelih ili odbijenih veza s jednog izvora.

Istovjetna nadzorna ploča izrađena je i za podsustav s Elasticsearchom, s istim prikazima, ali s upitima prilagođenima Elasticsearch jeziku upita. Budući da se prikazi i njihov raspored podudaraju, usporedba dvaju podsustava ne ovisi o načinu vizualizacije.

## 4.6. Automatska reakcija: `auto_reaction.py` i `nftables`

Za razliku od ostalih komponenata, skripta za automatsku reakciju `auto_reaction.py` izvodi se izravno na računalu domaćinu, a ne u kontejneru, jer mijenja njegova vatrozidna pravila i za to zahtijeva administratorske ovlasti. Skripta povezuje detekciju, odnosno Zeekova upozorenja [6], s odgovorom, tj. blokiranjem putem `nftables` kroz `denylist`.

Skripta prati najnovije retke datoteke `notice.log` naredbom `tail -F -n 0` gdje zastavica `-n 0` osigurava da se pri pokretanju zanemare postojeći zapisi, pa se reagira isključivo na nova upozorenja, a ne na povijest zapisanu prije pokretanja.

Za svako novo upozorenje skripta čita njegov tip (`note`) i izvorišnu adresu. Izvorišna se adresa preuzima iz polja `src`, a ako ono ne postoji, iz polja `id.orig_h` prema ispisu 4.17 Razlog je što vlastiti detektori postavljaju `$src` bez pridružene veze, dok upozorenja vezana uz konkretnu vezu nose `id.orig_h`.

```

note    = entry.get("note", "")
src_ip  = entry.get("src") or entry.get("id.orig_h", "")

if note in TARGET_NOTICES and src_ip and src_ip not in WHITELIST
:
    if src_ip not in BLOCKED_IPS:
        block(src_ip)    # nft add element ip diplomatski denylist
                           { src_ip }

```

**Kod 4.17.** Srž odlučivanja u `auto_reaction.py`

Odgovor se pokreće isključivo za unaprijed određen skup upozorenja (`TARGET_NOTICES`) kao pokušaji skeniranja portova, pogađanje SSH lozinke te podudaranja s indikatorima kompromitacije. Na taj način operator može smanjiti (ili povećati) broj upozorenja ovisno o situaciji i potrebama. Adresa usmjernika i adresa domaćina u redovnom radu nalaze se u listi dopuštenih adresa (`WHITELIST = "127.0.0.1", "10.0.1.1"`) kako se ne bi mogla blokirati vlastita infrastruktura.

Blokada se provodi dodavanjem izvorišne adrese u skup `denylist` unutar tablice `diplomski`. Sam skup i lanci koji odbacuju promet definirani su ranije, pri postavljanju mreže, u ispisu 4.6 Skripta mijenja isključivo elemente skupa, ne lance ni pravila. Zbog toga bilo kakav nenadani pad skripte ne ostavlja vatrozidna pravila bez nadzora, već se jezgra brine o tome da se pobrišu nakon zadanog vremena.

Trajanje blokiranja ne određuje skripta, nego se očitava iz trenutne (engl. *live*) konfiguracije samog skupa (funkcija `read_timeout`) kao jedinstvenog izvora istine. Istek pojedinog elementa potom obavlja jezgra operacijskog sustava na temelju vremenskog ograničenja skupa, neovisno o skripti. Posljedica je otpornost na nasilni prekid. Prekine li se skripta signalom `SIGKILL` ili ponestankom memorije, elementi svejedno isteknu sami, a cjelokupno se stanje uklanja brisanjem tablice. Skripta uz to vodi vlastitu evidenciju blokiranih adresa isključivo radi izbjegavanja ponovnog dodavanja iste adrese, dok zasebna dretva uklanja istekle zapise iz te evidencije. Blokada se bilježi tek ako je naredba `nft` uspješno izvršena, čime se izbjegava pogrešna evidencija nepostojeće blokade.

## 4.7. Izazovi pri implementaciji

Izgradnja sustava nije tekla bez zastoja. U nastavku se opisuju najznačajniji problemi uočeni tijekom implementacije i način na koji su riješeni, redom kako su se javljali kroz slojeve sustava.

Već pri postavljanju kontejnera utvrđeno je da ranije korištena slika `zeekur/zeek` više nije dostupna, pa je zamijenjena službenom slikom `zeek/zeek:8.0.6`. Verzija slike namjerno je fiksirana radi ponovljivosti; u produkcijskoj bi je primjeni trebalo redovito ažurirati na novija izdanja sa sigurnosnim zakrpama.

Na mrežnom sloju prvi je izazov bio sukob oko upravljanja sučeljem. `NetworkManager` preuzimao je sučelje `wlan1` i gasio pristupnu točku koju podiže `hostapd`. Problem je riješen izuzimanjem sučelja iz njegova nadzora naredbom `nmcli device set wlan1 managed no` prije konfiguracije pristupne točke [21].

Nakon što su `hostapd` i `dnsmasq` bili aktivni, a `wlan1` ispravno konfiguriran, klijenti i dalje nisu dobivali adresu. Pregledom `nft list ruleset` utvrđeno je da `ufw` presreće DHCP pakete (port 67/UDP) i odbacuje ih prije nego dopiju do `dnsmasq`-a. Problem je otklonjen iznimkom dopuštanja DHCP prometa i usmjeravanjem na sučelju `wlan1`, pravilima koja su potom ugrađena u skriptu `net_setReset_rules.sh` radi ponovljivosti.

Pokretanje `dnsmasq`-a isprva nije uspijevalo zbog zauzetosti porta 53 (`Address already in use`), koji je na povratnim adresama držao sistemski razlučivač `systemd-resolved`. Budući da `dnsmasq` ionako treba slušati samo na `wlan1`, sukob je otklonjen direktivom `bind-interfaces` kojom `dnsmasq` sluša isključivo na navedenom sučelju umjesto na svim adresama, bez gašenja sistemskog razlučivača. Uz to, prva inačica naziva mreže sadržavala je zgrade i razmake, koje `hostapd` odbija kao nevaljane znakove u SSID-u, pa je naziv prilagođen.

Na sloju ulaza zapisi pri prvom pokretanju Telegrafa nisu dospijevali u InfluxDB. Uzrok je bilo pogrešno raščlanjivanje Zeekovih ključeva. Format `json_v2` točku tumači kao razdjelnik putanje, dok je u nazivima poput `id.orig_h` ona dio samog ključa. Tek nakon izuzimanja točke u konfiguraciji (`id\.orig_h`) podatci su ispravno raščlanjeni i upisani. Dodatno, datoteka `conn.log` ne nastaje pri pokretanju Zeeka nego tek pri prvoj zabilježenoj

vezi, pa je Telegraf u međuvremenu prijavljivao njezino nepostojanje; problem se rješava sam čim kroz mrežu prođe promet.

U pogledu detekcije isprva je za skeniranje portova učitana ugrađena politika `policy/misc/scan`. U korištenoj inačici Zeeka je ta politika zastarjela i prestala je raditi, a prikladna zamjena uključena u Zeek instalaciju nije pronađena. Stoga je detekcija skeniranja preseljena u vlastiti detektor opisan ranije, čime je uklonjena ovisnost o zastarjeloj i nepodržanoj politici.

Na sloju vizualizacije uočeno je da nadzorne ploče na čistom postavljanju ne pronalaze svoj izvor podataka. Uzrok je u tome što Grafana izvoru bez izrijekom zadanog identifikatora dodjeljuje nasumičan `uid` [18], koji se onda ne podudara s identifikatorom zapisanim u izvezenoj nadzornoj ploči. Problem je riješen ručnim postavljanjem polja `uid` u opisu izvora podataka unutar `grafana/provisioning/datasources/influxdb.yaml` (i sličnoj putanji za Elasticsearch), čime se podudaranje osigurava pri svakom postavljanju.

Razlika između dvaju pozadinskih sustava očitovala se i pri vremenskim prikazima. Elasticsearch ograničava broj segmenata histograma (`search.max_buckets`, zadano 65536); fiksni interval poput jedne minute nad širokim vremenskim rasponom premašuje tu granicu, što se kod InfluxDB-a ne događa. Prikazi su prilagođeni korištenjem automatskog intervala koji se podešava prema odabranom rasponu. Ovo je ujedno konkretan primjer razlike u modelu dvaju sustava, detaljnije razmotrene u poglavlju evaluacije.

Naposljetku, pri opterećenju prometom generiranim alatom `t50` uočeno je da Zeek počinje gubiti pakete, što bilježi u zapisu `capture_loss.log`, kada dotok premaši približno pedeset tisuća paketa u sekundi. Za promet tipičan na lokalnoj mreži ciljanih razmjera to ne predstavlja ograničenje, no zabilježeno je kao gornja granica pouzdanog hvatanja na korištenom sklopovlju.

## 5. Evaluacija podsustava za pohranu

Cilj ovog poglavlja jest usporediti dva ispitana podsustava za pohranu opisana u poglavljima arhitekture i implementacije. Vremenski orijentiranu bazu InfluxDB s agentom Telegraf te dokumentno orijentiranu bazu Elasticsearch s agentom Filebeat. Usporedba se provodi nad istovjetnim ulaznim podacima kako bi se uočene razlike mogle pripisati isključivo pozadinskoj bazi i njezinu agentu, a ne ulazu. Naglasak je na trima mjerljivim svojstvima koja su za ciljanu primjenu (potrošački laptop ograničenih resursa) presudna - zauzeću prostora na disku, potrošnji radne memorije te kašnjenju upita.

### 5.1. Metodologija usporedbe

Mjerenja su provedena na domaćinu temeljenom na jezgri 7.0.1 carchos (x86\_64) s ukupno približno 12 GB dostupne radne memorije. Korištene su inačice InfluxDB 2.7, Elasticsearch 8.17 i Grafana 11.4, dok je veličina JVM gomile Elasticsearcha ograničena postavkom `ES_JAVA_OPTS=-Xms512m` odnosno 512 MB.

Za razliku od produkcijske konfiguracije, u kojoj se zbog sukoba na portu 3000 (Grafana) i istovjetnog naziva senzorskog kontejnera izvodi isključivo jedan podsustav, mjerenja su provedena uz istovremeno podignute obje baze i pripadne agente, no bez Grafane i bez senzora. Time se obje baze izlažu istom opterećenju domaćina u istom trenutku, pa su mjerenja međusobno usporediva pod identičnim uvjetima. Kao ulaz korišten je jedan zamrznuti zapis `conn.log` veličine 4.7 MB s 14027 zapisa veza, koji oba agenta čitaju iz istoga dijeljenog direktorija.

Poštenost usporedbe počiva na tome da oba podsustava obrade isti ulaz, svaki ga pročitavši točno jednom, a to se provjerava brojem zapisa. Elasticsearch indeksira svaki redak

kao zaseban dokument, pa broj dokumenata mora odgovarati broju redaka ulaza. InfluxDB, naprotiv, točku jednoznačno određuje kombinacijom mjerenja, skupa oznaka i vremenske oznake. Zapisi s istovjetnim vrijednostima tih polja stapaju se u jednu točku umjesto da se pišu zasebno. Posljedica je da InfluxDB prijavljuje nešto manji broj točaka od broja ulaznih redaka, što nije gubitak podataka nego svojstvo modela pohrane i podrobnije se razmatra u raspravi. Izmjereni su brojevi prikazani u tablici 5.1.

| Izvor                       | Broj zapisa |
|-----------------------------|-------------|
| Redaka u conn.log (ulaz)    | 14027       |
| Elasticsearch dokumenata    | 14027       |
| InfluxDB točaka (zeek_conn) | 13435       |

Tablica 5.1. Provjera istovjetnosti ulaza

Mjerenja su automatizirana skriptom `benchmark.sh`, koja redom provodi provjeru ulaza, mjerenje prostora, uzorkovanje memorije i mjerenog kašnjenja upita.

## 5.2. Potrošnja prostora na disku

Mjeri se zauzeće na disku za istovjetan skup Zeekovih zapisa, uz izvornu datoteku `conn.log` kao referentnu vrijednost. Za InfluxDB mjeri se veličina podatkovnog direktorija pripadnog spremnika (engl. *bucket*), a za Elasticsearch veličina indeksa `zeek-conn` dobivena iz sučelja `_cat/indices`.

| Izvor                          | Zauzeće |
|--------------------------------|---------|
| Sirovi conn.log referenca      | 4.7 MB  |
| Elasticsearch indeks zeek-conn | 4.8 MB  |
| InfluxDB spremnik zeek-logs    | 1.5 MB  |

Tablica 5.2. Zauzeće prostora na disku

Kao dodatni kontekst pri tumačenju ovih vrijednosti potrebno je naglasiti veličinu izmjerenog skupa. Drugim riječima izmjereni skup je malen (reda veličine nekoliko megabajta), pa razlike u zauzeću prostora prouzročavaju ponajprije dodatni troškovi formata pohrane, a ne ponašanje pri rastu količine podataka. Elasticsearch gradi invertirani indeks (uz pohranu podataka) koji radi pretrage po sadržaju polja, što povećava zauzeće diska u odnosu na sam volumen ulaza. InfluxDB, naprotiv, primjenjuje kompresiju prilagođenu vremenskim nizovima, pa je izmjereni spremnik (1.5 MB) zauzeo otprilike trećinu Elasticsearchova indeksa

i manje od same sirove datoteke. Posljedice tih značajki baza detaljnije se razmatraju u raspravi.

### 5.3. Potrošnja radne memorije

Potrošnja radne memorije mjerena je po kontejneru, u tri uzorka nakon što se sustav prilagodio, korištenjem Docker statistike. Tablica 5.3. prikazuje reprezentativne vrijednosti.

| Kontejner                         | Radna memorija |
|-----------------------------------|----------------|
| InfluxDB (baza, podsustav A)      | 106 MB         |
| Telegraf (agent, podsustav A)     | 40 MB          |
| Elasticsearch (baza, podsustav B) | 576 MB         |
| Filebeat (agent, podsustav B)     | 84 MB          |

Tablica 5.3. Potrošnja radne memorije po kontejneru (nakon prilagodbe)

Razlika u potrošnji memorije najizraženija je i najpostojanija od svih mjerenih veličina. Cijeli podsustav s InfluxDB-om (baza i agent) zadržava se na razini oko 146 MB, dok podsustav s Elasticsearchom (baza i agent) troši oko 660 MB, odnosno približno četiri i pol puta više. Bitno je da ta razlika nije posljedica količine podataka, nego donje granice koju nameće Java virtualni stroj. Potrošnja Elasticsearcha određena je prije svega zadanom veličinom JVM gomile (ovdje 512 MB), koja predstavlja donju granicu neovisnu o broju pohranjenih zapisa. Uz veću gomilu otisak bi bio i veći. Za ciljanu primjenu na sklopovlju ograničenih resursa ta je donja granica presudna i razmatra se u zaključku. Valja napomenuti da je ta granica pomična: što se više memorije dodijeli, pretraživanja su brža.

### 5.4. Kašnjenje upita

Mjereno je kašnjenje triju upita koje koristi i nadzorna ploča: brojanje veza, ljestvica deset najčešćih odredišnih adresa te agregacija broja veza po vremenskim prozorima. Svaki je upit pokrenut deset puta, uz odbacivanje dvaju početnih pokretanja radi zagrijavanja, a prijavljeni su medijan te najmanja i najveća izmjerena vrijednost. Mjeri se ukupno kašnjenje kakvo klijent vidi na povratnoj adresi, dakle uključujući obradu zahtjeva, a ne samo unutarnje vrijeme izvođenja upita u jezgri baze. Kako bi usporedba bila poštena, brojanje u oba sustava obavlja prolaz kroz zapise (u Elasticsearchu agregacijom `value_count`, a ne očitanjem održavanoga broja dokumenata).

| Upit                      | InfluxDB (Flux)  | Elasticsearch |
|---------------------------|------------------|---------------|
| Brojanje (sken)           | 49,5 (45,3-51,9) | 4,4 (4,0-5,3) |
| Deset najčešćih odredišta | 42,4 (40,3-44,5) | 5,1 (4,4-5,3) |
| Agregacija po minutama    | 63,6 (62,8-64,9) | 7,4 (6,1-9,1) |

Tablica 5.4. Kašnjenje upita: medijan (najmanje-najveće), u milisekundama

Na svim trima upitima Elasticsearch postiže niže kašnjenje. Rezultat je nužno tumačiti uz činjenicu da je izmjereni skup malen, pa se mjeri pretežno dodatni trošak izvođenja upita (priprema Flux upita i obrada zahtjeva nad sučeljem InfluxDB-a naspram Luceneova izvođenja u Elasticsearchu), a ne ponašanje pri rastu količine podataka. Apsolutne su vrijednosti u oba sustava niske i za potrebe osvježavanja nadzorne ploče u sekundnim razmacima posve dostatne; razlika bi mogla dobiti na značaju tek pri znatno većim skupovima ili pri upotrebi za pravovremenu reakciju, što ostaje smjernicom za daljnji rad.

## 5.5. Rasprava rezultata

Mjerenja se mogu objediniti kroz razliku u modelu podataka dvaju sustava, opisanu u poglavlju o teorijskim osnovama. Elasticsearch svaki Zeekov zapis tretira kao dokument i nad poljima gradi invertirani indeks, što mu daje snažnu pretragu punim tekstom, povoljno kašnjenje agregacijskih upita i korelaciju izvora, ali po cijeni većega prostornog otiska i, presudno, donje granice radne memorije koju nameće JVM. InfluxDB je optimiziran za vremenske nizove pa mu je vrijeme glavna organizacijska dimenzija. Također primjenjuje kompresiju i politike zadržavanja, a memorijski otisak ostaje malen. Iz istoga modela slijedi i opažena razlika u broju zapisa iz tablice 5.1. Budući da InfluxDB točku određuje skupom oznaka i vremenskom oznakom, veze koje dijele te vrijednosti stapaju se u jednu točku, pa je broj točaka manji od broja ulaznih redaka. Stapanje pogađa isključivo agregatni broj zapisa veza (otprilike 4%) i posljedica je svjesne odluke da se identifikator veze uid drži poljem, a ne oznakom - držanje oznakom očuvalo bi svaki redak, ali bi količina vremenskih serija narasla na broj veza i poništila memorijsku prednost zbog koje se InfluxDB i odabire. Zapisi notice nisu zahvaćeni. Za primjenu usmjerenu na vizualizaciju obrazaca i dojavu, a ne na forenzičku rekonstrukciju pojedine veze, riječ je o prihvatljivom kompromisu. Unutar Elasticsearcha, gdje je svaki redak zaseban dokument, takvoga stapanja nema.

Posljedica je očita specijalizacija alata. InfluxDB je bolji izbor kada su upiti temeljeni na

vremenu i kad su računalni resursi domaćina oskudni. Kad su potrebni pretraga po sadržaju polja (primjerice traženje domena u `dns.log`), niže kašnjenje složenijih agregacija i korelacija više vrsta zapisa u jednom upitu Elasticsearch se ističe kao očiti odabir. InfluxDB ne podržava prirodno te mogućnosti te zahtijeva agregaciju na strani klijenta.

Naposljetku valja istaknuti glavno ograničenje cijele evaluacije. Mjerenja su provedena na malenom skupu podataka, pa pouzdano govore o fiksnim zapisima i donjim granicama potrošnje, ali ne i o ponašanju sustava pri znatnom rastu količine prometa. Mjerenje na većem skupu ostavlja se kao smjernica za daljnji rad iako se može primijetiti trend gdje bi Elasticsearch bio puno bolja opcija.

## 5.6. Zaključak usporedbe

Za ciljanu primjenu ovog rada, nadzor lokalne mreže na potrošačkom sklopovlju ograničenih resursa, odlučujućom se pokazuje potrošnja radne memorije. Podsustav s InfluxDB-om zadržava se na razini stotinjak megabajta, dok podsustav s Elasticsearchom troši višestruko više, ponajprije zbog donje granice koju nameće JVM gomila. Ta je granica neovisna o količini pohranjenih podataka, pa Elasticsearch i pri zanemarivu opterećenju zauzima više memorije nego cijeli InfluxDB podsustav pod radnim opterećenjem; na domaćinu koji istovremeno obavlja uloge usmjernika, pristupne točke i senzora upravo je taj fiksni prag presudan. Tu je granicu moguće sniziti postavkom veličine JVM gomile, no na uštrb brzine i stabilnosti pretraživanja pri većim opterećenjima, čime slabi upravo prednost zbog koje bi se Elasticsearch inače odabrao.

Na svim trima izmjerenim upitima Elasticsearch postiže niže kašnjenje, kako prikazuje tablica 5.4. Apsolutne su vrijednosti obaju sustava ipak reda nekoliko desetaka milisekundi i za osvježavanje nadzorne ploče u sekundnim razmacima zanemarive, pa ne mijenjaju odluku koju daje memorijski otisak. Zauzeće diska na izmjerenom je skupu premaleno da bi samo po sebi bilo argumentom te se u ovom zaključku ne uzima kao odlučujuće.

Stoga se za produkcijsku konfiguraciju, u skladu s najavom iz poglavlja arhitekture da se odabire jedan od dvaju podsustava, bira InfluxDB. Time se ujedno pokazuje da je usporedivu temeljnu funkcionalnost moguće postići uz memorijski otisak reda veličine stotinjak megabajta, znatno ispod zahtjeva sveobuhvatnih rješenja poput Security Oniona navedenih u

poglavlju o teorijskim osnovama.

Elasticsearch ostaje prikladan kad su potrebni pretraga po sadržaju polja (primjerice traženje domena u `dns.log`), niže kašnjenje složenijih agregacija i korelacija više vrsta zapisa u jednom upitu. Te prednosti slijede iz modela podataka, no nisu zasebno izmjerene u ovom radu jer mjerenja obuhvaćaju samo `conn.log` te se njihova kvantifikacija ostavlja kao smjernica za daljnji rad.

## 6. Testiranje detekcije i automatske reakcije

Cilj je ovog poglavlja funkcionalno provjeriti dvije sposobnosti sustava koje nisu obuhvaćene usporedbom podsustava za pohranu iz prethodnog poglavlja. Sposobnost prepoznavanja sumnjivog ponašanja u prometu te sposobnost automatskog odgovora na prepoznatu prijetnju. Provjera se provodi kao niz nadziranih testova u kojima se s poznatog klijenta izaziva određeni obrazac prometa, a potom se promatra cijeli lanac od Zeekova upozorenja u `notice.log`, preko skripte `auto_reaction.py`, do izmjene elemenata skupa `denylist` u tablici `diplomski`. Svaki se test prati po izvorišnoj adresi, čime se uspostavlja jednoznačna veza između izazvanog događaja, zabilježenog upozorenja i poduzete reakcije. Navode se isključivo stvarno izmjereni zapisi.

### 6.1. Testno okruženje i metodologija

Testovi su provedeni na sklopovlju opisanom u poglavlju arhitekture. Prijenosno računalo istovremeno obavlja uloge usmjernika i bežične pristupne točke, pri čemu je sučelje `enp2s0` povezano prema vanjskoj mreži (`eduroam`), a sučelje `wlan1` podiže pristupnu točku za lokalnu podmrežu `10.0.1.0/24`. Postavljene su dvije jasno razdvojene uloge:

- **Napadač** — klijent priključen na pristupnu točku. U svim je testovima detekcije korišten mobilni uređaj s adresom `10.0.1.94`.
- **Branitelj i senzor** — samo prijenosno računalo (`10.0.1.1`), na kojem rade Zeek, `nftables` i skripta `auto_reaction.py`.

Zeek promet hvata na sučelju `wlan1`, dakle prije prevođenja mrežnih adresa, pa izvorišne adrese ostaju stvarne adrese uređaja u lokalnoj mreži - ta je činjenica ključna jer omogućuje pripisivanje upozorenja pojedinom uređaju.

Za izazivanje pojedinih obrazaca prometa korišteni su sljedeći alati: `nmap` za skeniranje portova, `sshpass` za automatske ponavljane pokušaje prijave putem SSH, `curl` za uspostavljanje veze alatom prema adresi iz datoteke `intel.dat` za podudaranje s indikatorom kompromitacije te `hping3` za lažiranje izvorišne adrese pri ispitivanju ograničenja.

Reakcija se temelji na sljedećem ponašanju skripte: po prepoznavanju obavijesti izvorišna adresa (`src`) dodaje se kao element skupa `denylist` s vremenskim ograničenjem od 30 sekundi, osim ako se nalazi na popisu dopuštenih adresa. U redovnom radu popis dopuštenih adresa sadrži 127.0.0.1 i 10.0.1.1; adresa 10.0.1.94 dodana je na taj popis isključivo za potrebe testiranja.

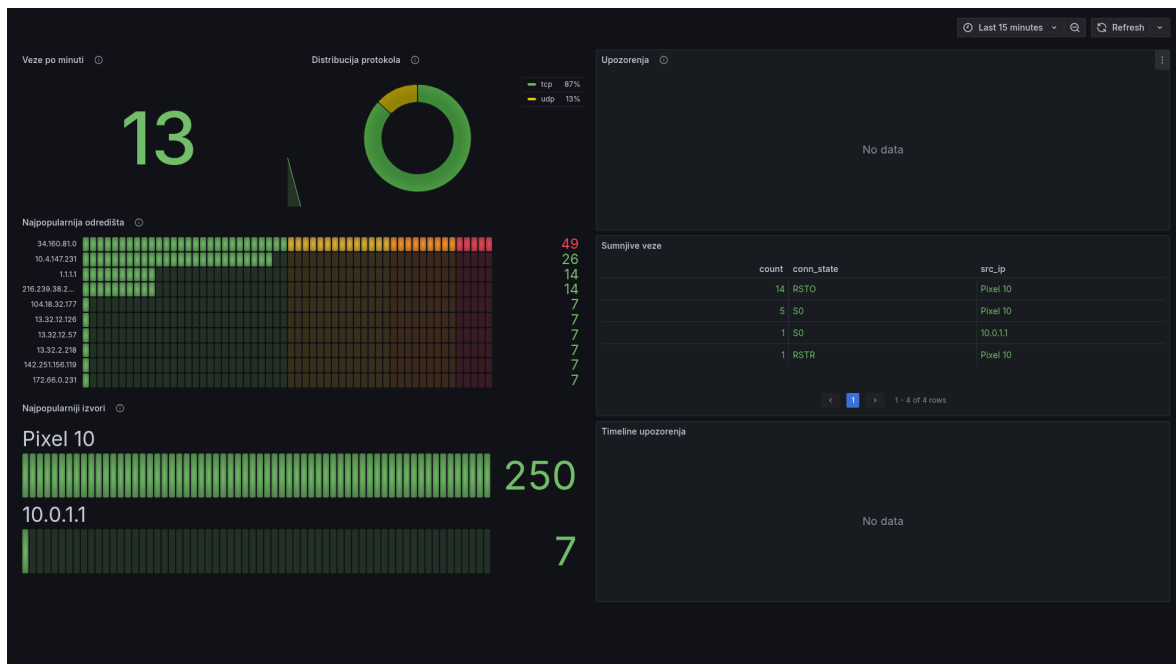
Radi provjere lažno pozitivnih odziva kroz pristupnu je točku propušten uobičajen promet (pregledavanje weba, dohvaćanje datoteka, ažuriranja sustava) tijekom 3 sata 38 minuta 41 sekunde. Nijedan obrazac nije premašio pragove definirane u `local.zeek` (50 različitih portova u pet minuta odnosno 15 SSH veza), pa skup `denylist` nije popunjen niti je jedan legitiman uređaj blokiran.

## 6.2. Provjera toka podataka

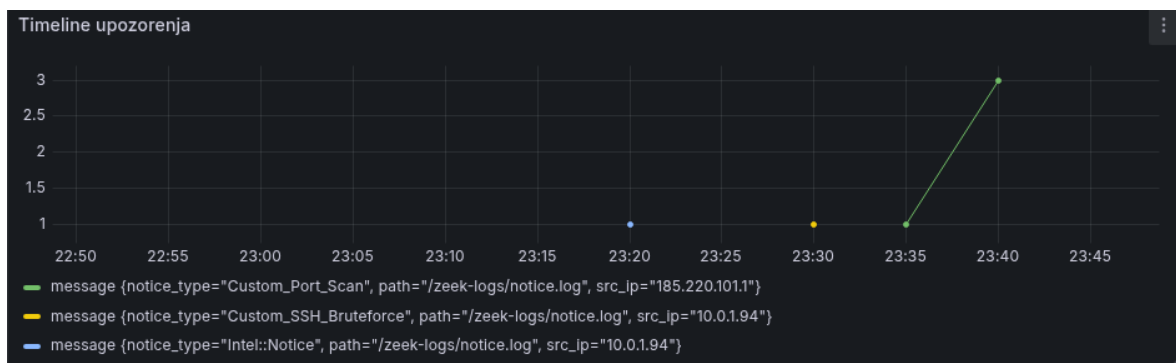
Prije ispitivanja detekcije potvrđuje se da podatci prolaze cijelim lancem obrade. Na klijentu se uspostavi veza, nakon čega se utvrđuje da je nastao zapis u `conn.log`, da ga je preuzeo agent (Telegraf odnosno Filebeat), da je pohranjen u pripadnoj bazi te da je vidljiv na odgovarajućem prikazu u Grafani. Na nadzornoj ploči, na slici 6.1., izravno se očitava da podatci pristižu - prikazani su broj veza u minuti, raspodjela protokola te ljestvice najčešćih izvorišnih i odredišnih adresa, u kojima se pojavljuju stvarne adrese klijenata iz lokalne pod mreže. Time je potvrđeno da je tok podataka cjelovit i da se izvorišne adrese očuvane na sučelju `wlan1` ispravno prenose do prikaza.

## 6.3. Testiranje detekcije

Provedena su tri testa detekcije: skeniranje portova, bruteforce SSH prijave i podudaranje s indikatorom kompromitacije. Svi su izvedeni s klijenta 10.0.1.94, osim lažiranog skeniranja iz odjeljka o ograničenjima. Sva tri tipa upozorenja vidljiva su na vremenskoj crti nadzorne ploče na slici 6.2., s pripadnim izvorišnim adresama.



**Slika 6.1.** Prikaz podataka na nadzornoj ploči: broj veza u minuti, raspodjela protokola te najčešći izvori i odredišta; u promatranom razdoblju nema upozorenja (osnovno stanje prometa)



**Slika 6.2.** Vremenska crta upozorenja: Custom\_Port\_Scan (185.220.101.1) te Custom\_SSH\_Bruteforce i Intel::Notice (oba 10.0.1.94)

### 6.3.1. Skeniranje portova

Skeniranje je izazvano s klijenta 10.0.1.94 naredbom `nmap -Pn -T4 10.0.1.1`. Prekoračenjem praga od 50 različitih kontaktiranih portova prema istom cilju, Zeek je zabilježio upozorenje tipa Custom\_Port\_Scan sa stvarnom izvorišnom adresom 10.0.1.94 u ispisu 6.1 Detektor djeluje na događaju `new_connection`, pa reagira na pokušaj veze i ne ovisi o njezinom uspješnom uspostavljanju.

```
{ "ts": 1781467564.943518, "note": "Custom_Port_Scan",
  "msg": "Moguci port scan: 10.0.1.94 kontaktirao 50 razlicitih
        portova na 10.0.1.1",
  "src": "10.0.1.94", "actions": ["Notice::ACTION_LOG"],
  "email_dest": [], "suppress_for": 30.0 }
```

### 6.3.2. Bruteforce SSH prijave

Bruteforce je izveden s klijenta 10.0.1.94 kao 40 uzastopnih neuspjelih pokušaja prijave putem SSH (alat sshpass) prema domaćinu. Korišteni prilagođeni detektor broji *učestalost veza* prema portu 22, neovisno o ishodu autentikacije, te je komplementaran Zeekovu detektoru SSH::Password\_Guessing koji radi na temelju zaključene autentikacije. Pri pragu od 15 veza zabilježeno je upozorenje tipa Custom\_SSH\_Bruteforce; tijekom 40 pokušaja istek prozora suzbijanja (30 s) doveo je do druge obavijesti pri 36 veza 6.2

```
{"ts":1781468972.322033,"note":"Custom_SSH_Bruteforce",
"msg":"Moguci SSH bruteforce: 10.0.1.94 otvorio 15 konekcija
      prema portu 22 na 10.0.1.1",
"src":"10.0.1.94","suppress_for":30.0}
{"ts":1781469003.721936,"note":"Custom_SSH_Bruteforce",
"msg":"Moguci SSH bruteforce: 10.0.1.94 otvorio 36 konekcija
      prema portu 22 na 10.0.1.1",
"src":"10.0.1.94","suppress_for":30.0}
```

**Kod 6.2.** Dvije obavijesti o SSH bruteforceu tijekom istog niza pokušaja (notice.log)

Obavijesti su vidljive i na nadzornoj ploči, na slici 6.3., gdje je za izvor 10.0.1.94 zabilježeno više detekcija Custom\_SSH\_Bruteforce pri pragovima od 15 te 35-36 veza.

| _time               | Upozorenje                                                              | Tip                | Izvor     |
|---------------------|-------------------------------------------------------------------------|--------------------|-----------|
| 2026-06-14 22:39:35 | Moguci SSH bruteforce: 10.0.1.94 otvorio 35 konekcija prema portu 22 na | Custom_SSH_Brutefo | 10.0.1.94 |
| 2026-06-14 22:38:54 | Moguci SSH bruteforce: 10.0.1.94 otvorio 15 konekcija prema portu 22 na | Custom_SSH_Brutefo | 10.0.1.94 |
| 2026-06-14 22:30:03 | Moguci SSH bruteforce: 10.0.1.94 otvorio 36 konekcija prema portu 22 na | Custom_SSH_Brutefo | 10.0.1.94 |
| 2026-06-14 22:29:32 | Moguci SSH bruteforce: 10.0.1.94 otvorio 15 konekcija prema portu 22 na | Custom_SSH_Brutefo | 10.0.1.94 |

**Slika 6.3.** Upozorenja o SSH bruteforceu na nadzornoj ploči (izvor 10.0.1.94)

### 6.3.3. Podudaranje s indikatorom kompromitacije

Podudaranje je izazvano uspostavljanjem veze s klijenta 10.0.1.94 prema adresi unaprijed unesenoj u datoteku intel.dat. Putem okvira (engl. *Intel framework*) (skripte intel/seen i intel/do\_notice) Zeek poklapa adrese na događaju uspostavljene veze

(connection\_established); za test je stoga ključno da se veza *uspostavi*, neovisno o isходу na aplikacijskoj razini. Veza je rezultirala neuspjehom pri (engl. *TLS*) pregovaranju, ali je TCP rukovanje dovršeno, što je dovoljno za poklapanje. Zeek je zabilježio upozorenje tipa Intel::Notice u ispisu 6.3

```
{"ts":1781471801.228362,"id.orig_h":"10.0.1.94","id.resp_h":
  "172.66.147.243",
  "id.resp_p":443,"proto":"tcp","note":"Intel::Notice",
  "msg":"Intel hit on 172.66.147.243 at Conn::IN_RESP",
  "src":"10.0.1.94","dst":"172.66.147.243","p":443,"
  suppress_for":3600.0}
```

**Kod 6.3.** Upozorenje o podudaranju s indikatorom kompromitacije (notice.log)

Za testni je indikator korištena dostupna javna adresa, što je uobičajena i sigurna praksa pri provjeri Intel okvira. Vrijednost `suppress_for` ovdje iznosi 3600 sekundi (zadani interval suzbijanja obavijesti); taj se interval prema potrebi snižava postavkom u `local.zeek`, čime se trguje između količine obavijesti i odzivnosti te je on odvojen od `TIMEOUT` kojeg se postavlja za duljinu trajanja blokade unutar nft skupa `denylist` u tablici `diplomski`.

## 6.4. Testiranje automatske reakcije

### 6.4.1. Blokiranje izvora i istek blokade

Reakcija je promatrana na primjeru skeniranja portova iz ispisa 6.1 Prije testa skup `denylist` bio je prazan; ubrzo nakon detekcije izvorišna se adresa nalazi u skupu, uz preostalo vrijeme do isteka koje očitava jezgra u ispisu 6.4 Skripta `auto_reaction.py` pritom je ispisala odgovarajuću potvrdu u ispisu 6.5

```
# prije:
table ip diplomski {
    set denylist { type ipv4_addr ; timeout 30s }
}
# tijekom blokade:
table ip diplomski {
    set denylist {
        type ipv4_addr
        timeout 30s
        elements = { 10.0.1.94 expires 22s818ms }
    }
}
```

**Kod 6.4.** Skup `denylist` prije i tijekom blokade

```
[!] Detektiran Custom_Port_Scan.
[!] Blokirani IP: 10.0.1.94
```

```
[+] Istekla evidencija blokade: 10.0.1.94
```

**Kod 6.5.** Ispis skripte `auto_reaction.py`

Trajanje blokade određeno je vremenskim ograničenjem skupa (`timeout 30s`). Nakon isteka blokade jezgra automatski uklanja element, bez intervencije skripte ili operatora. To je moguće potvrditi porukom o isteku u svakom testu te ponovnim pregledom praznog skupa.

Za odlaznu vezu prema indikatoru kompromitacije skripta blokira polje `src` iz obavijesti, a to je *unutarnji* klijent koji je ostvario vezu (10.0.1.94), a ne vanjska maliciozna adresa. Riječ je o namjernom postupku. Vanjska je adresa izvan dosega upravljanja, dok se blokiranjem unutarnjeg domaćina ograničava daljnja komunikacija potencijalno kompromitiranog uređaja.

Blokada je potvrđena i pregledom skupa nakon podudaranja s indikatorom kompromitacije. U skupu `denylist` nalazi se unutarnji klijent 10.0.1.94, a ne vanjska adresa indikatora što je vidljivo u ispisu 6.6

```
table ip diplomski {  
    set denylist {  
        type ipv4_addr  
        timeout 30s  
        elements = { 10.0.1.94 expires 11s813ms }  
    }  
}
```

**Kod 6.6.** Skup `denylist` nakon podudaranja s indikatorom kompromitacije

## 6.4.2. Zaštita vlastite infrastrukture popisom dopuštenih adresa

Provjereno je da skripta ne reagira kad je izvorišna adresa na popisu dopuštenih. U testu je adresa 10.0.1.94 privremeno dodana na popis, nakon čega je ponovljen SSH brute force. Zeek je i dalje zabilježio upozorenje `Custom_SSH_BruteForce` (detekcija nije izostala), no skup `denylist` ostao je prazan, a skripta nije poduzela reakciju, što je vidljivo u ispisu 6.7. Time je potvrđeno da popis dopuštenih adresa sprječava blokiranje povjerljivih izvora bez utjecaja na detekciju.

```
# WHITELIST = {"127.0.0.1", "10.0.1.1", "10.0.1.94"}  
[*] Nadzor pokrenut nad zeek-logs/notice.log  
# upozorenje se nalazi u notice.log, ali skripta nije  
    reagirala  
# skup denylist ostao prazan  
# sustav radi kako bi i trebao
```

## 6.5. Ograničenja uočena testiranjem

### 6.5.1. Lažiranje izvorišne adrese

Reakcija se temelji na izvorišnoj adresi iz obavijesti, pa je njezina ispravnost ograničena ispravnošću te adrese. Ograničenje je provjereno namjernim lažiranjem izvorišne adrese alatom hping3, kojim je izvedeno skeniranje portova uz krivotvorenu adresu 185.220.101.1 (hping3 -S --spoof 185.220.101.1 --scan 1-1000 10.0.1.94). Zeek je zabilježio upozorenje s krivotvorenom adresom, a skripta je upravo nju dodala u skup denylist u ispisu 6.8

```
# notice.log:  "note":"Custom_Port_Scan", "src
               ":"185.220.101.1"
table ip diplomski {
    set denylist {
        type ipv4_addr
        timeout 30s
        elements = { 185.220.101.1 expires 19s395ms
                    }
    }
}
```

**Kod 6.8.** Ispis denylist

Potvrđeno je očekivano, ali nepoželjno ponašanje. Sustav blokira adresu navedenu u obavijesti, koja ne mora odgovarati stvarnom napadaču. Stvarni napadač ostaje neblokirani, a lažira li adresu legitimnog uređaja koji nije na popisu dopuštenih, sustav će blokirati taj uređaj. Popis dopuštenih adresa tu ne pomaže jer štiti samo usmjernik i lokalnog domaćina, a ne i proizvoljne uređaje od povjerenja; štoviše, oslanja se na izvorišnu adresu koja je i sama krivotvorljiva. Djelotvorno ublažavanje je filtriranje na ulazu sučelja wlan1 (odbacivanje paketa čija izvorišna adresa nije iz 10.0.1.0/24), čime se krivotvoreni paket odbacuje prije detekcije.

Valja razlučiti i opseg sposobnosti. Detekcija i reakcija djeluju nad prometom koji Zeek vidi na sučelju wlan1; nepozvani dolazni promet iz širega područja mreže na sučelju enp2s0 odbacuje se zadanom politikom vatrozida ufw i nije zadaća ovoga sloja.

### **6.5.2. Otpornost na nasilni prekid skripte**

Budući da trajanje blokade nameće vremensko ograničenje skupa, a ne sama skripta, nasilni prekid skripte ne ostavlja zaostala vatrozidna pravila. Prekine li se skripta, elementi skupa svejedno isteknu djelovanjem jezgre, što je u svakom testu potvrđeno automatskim isticanjem blokade. Time se izbjegava slabost izvedbi u kojima prekid posrednika ostavlja mrežu u nedefiniranom stanju. Podrobnije obrazloženje dano je u poglavlju implementacije.

### **6.5.3. Izloženost nadzornih servisa**

Tijekom skeniranja uočeno je da je na domaćinu 10.0.1.1 otvoren port 8086 (InfluxDB), dohvatljiv iz nadziranog segmenta. Nadzorni servisi (InfluxDB, Grafana) time čine dio površine napada vidljive klijentima, pa bi ih u stvarnoj primjeni trebalo izolirati od nadziranog segmenta. U radu je to kratko popravljeno ovom linijom u `docker-compose.yml`: "127.0.0.1:8086:8086" za InfluxDB odnosno "127.0.0.1:3000:3000" za Grafanu. Svi načini testiranja i mjerenja se mogu naći u dodatku `benchmark.sh`.

## 7. Zaključak i smjernice za daljnji rad

U radu je osmišljen i implementiran središnji sustav za nadzor i vizualizaciju komunikacijskih obrazaca lokalne mreže. Sustav pretvara uređaj s operacijskim sustavom Linux u jedinstveni operacijski centar koji istovremeno obavlja uloge usmjernika, bežične pristupne točke, mrežnog senzora i vatrozida. Sav se promet lokalne mreže usmjerava kroz jednu točku i time postiže potpuna vidljivost komunikacije. Promet se analizira Zeekom na sučelju prije prevođenja adresa, pohranjuje u jedan od dvaju ispitanih podsustava i vizualizira u Grafani, dok se na temelju Zeekovih upozorenja provodi automatsko blokiranje izvorišnih adresa.

Glavnu ulogu u prilagodljivosti sustava ima Zeekov skriptni jezik. Detekcijska logika nije ograničena na konačan skup potpisa, nego se definira u datoteci `local.zeek` bez izmjene jezgre sustava, pa se pragovi, vremenski prozori i suzbijanje obavijesti prilagođavaju specifičnostima nadzirane mreže. Uz pomoć skriptnog jezika su napisani vlastiti detektori skeniranja portova i učestalosti SSH veza, dok su se indikatori kompromitacije priključili koristeći Intel okvir koji funkcionira neovisno o detekcijskoj logici. Upravo skriptni jezik, a ne pojedini ugrađeni mehanizam, čini sustav proširivim i prenosivim na druge obrasce prijetnji.

Naglasak rada nije na minimalnosti sklopovlja na kojem se rad izvodi, nego na cjelovitosti i automatiziranosti lanca detekcije i reakcije. Zabilježen je i opisan put od opažanja sumnjivih obrazaca ponašanja, evidencije tih obrazaca kroz `notice.log` zapis i reakcije kroz `auto_reaction.py` do izmjene elemenata u skupu `denylist` unutar `nftables` tablice koje se događa bez intervencije operatora. Dodatno, budući da trajanje blokade nameće vremensko ograničenje skupa kojim upravlja jezgra, a ne sama skripta, sustav je otporan na neplanirani prekid. Kontejnerizacija servisa i razdvajanje prikupljanja od pohrane montiranjem dijeljenog direktorija isključivo za čitanje čine arhitekturu prenosivom i neovisnom o konkretnom domaćinu. Demonstracija na potrošačkom prijenosnom računalu stoga poka-

zuje donju granicu zahtjeva sustava, a ne gornju granicu njegove primjenjivosti - isti se stog može prenijeti na snažniji domaćin bez izmjene detekcijske odnosno reakcijske logike pod uvjetom da se koristi Linux.

U skladu s time treba tumačiti i odabir baze InfluxDB iz poglavlja evaluacije. Taj odabir vrijedi za demonstrirani profil primjene, u kojem domaćin ograničenih resursa istovremeno obavlja više uloga, pa donja granica radne memorije postaje presudna. Budući da je sloj pohrane odvojen od prikupljanja, zamjena pozadinske baze drugim ispitanim podsustavom - Elasticsearchom, kada su presudni pretraga po sadržaju polja i korelacija više vrsta zapisa - svodi se na izmjenu konfiguracije, a ne na preinaku sustava. Mogućnost odabira između dvaju podsustava zato nije ograničenje, nego svojstvo arhitekture.

Smjernice za daljnji rad proizlaze iz ograničenja evidentiranih ovim radom. Mjerenja podsustava za pohranu provedena su na malenom skupu podataka, pa pouzdano govore o donjim granicama potrošnje, ali ne i o ponašanju pri znatnom rastu količine prometa. Mjerenja na većem skupu podataka, na kojima se očekuje izraženija prednost Elasticsearcha, ostaju prijedlog budućeg istraživačkog rada. Reakcija se oslanja na izvorišnu adresu iz zapisa upozorenja što predstavlja problem ako napadač krivotvori adresu izvora. Kao ublažavanje ovog problema predlaže se filtriranje na ulazu sučelja pristupne točke čime se odbacuju paketi izvan dodijeljenog adresnog prostora prije same detekcije (a posljedično i reakcije). Vidljivost prometa zaustavlja se pred šifriranim DNS-om (DoH i DoT), što ostaje poznata slijepa točka pasivnog nadzora. Detekcijsku je logiku moguće proširiti na dodatne vrste zapisa koje su u agentima pripremljene, ali namjerno isključene radi poštene usporedbe podsustava, pri čemu upravo skriptni jezik omogućuje izgradnju novih detektora nad tim zapisima. Dosadašnji se detektori oslanjaju na unaprijed zadane pragove i vremenske prozore, pa prepoznaju isključivo obrasce predviđene pravilima. Kako bi se izbjegle mane statičke detekcije, trenutni je stog moguće proširiti slojem koji obrasce prepoznaje uz pomoć modela strojnoga odnosno dubokoga učenja. Takav bi se model trenirao nad metapodacima koje Zeek prikuplja te bi, na temelju pojedinačnih veza ili skupina veza, zaključivao je li komunikacija zlonamjerna. Obrada bi se provodila na zahtjev operatora ili povremenim, nasumičnim uzorkovanjem parova adresa i njihove komunikacije, kako se resursi sustava ne bi nepotrebno trošili.

Naposljetku, zabilježena gornja granica pouzdanog hvatanja na korištenom sklopovlju ostaje smjernica za ispitivanje skaliranja, kao i razmatranje uporabe niskog kašnjenja upita

za pravovremenu reakciju umjesto isključivo za osvježavanje nadzorne ploče.

## Literatura

- [1] Splunk Inc., “Splunk enterprise documentation”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://docs.splunk.com/Documentation/Splunk>
- [2] Security Onion Solutions, “Security onion — intrusion detection, enterprise security monitoring, and log management”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://securityonionsolutions.com/software>
- [3] R. Bejtlich, *The Practice of Network Security Monitoring*. No Starch Press, 2013.
- [4] J. Malinen, “hostapd: Ieee 802.11 ap, ieee 802.1x/wpa/wpa2/eap/radius authenticator”, 2024., pristupljeno: 13.3.2026. [Mrežno]. Adresa: <https://w1.fi/hostapd/>
- [5] S. Kelley, “dnsmasq — a lightweight dhcp and caching dns server”, 2024., pristupljeno: 13.3.2026. [Mrežno]. Adresa: <https://dnsmasq.org>
- [6] The Zeek Project, “Zeek documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.zeek.org>
- [7] T. O. I. S. F. (OISF), “Suricata: Doxygen documentation”, 2023., pristupljeno: 10.7.2026. [Mrežno]. Adresa: <https://doxygen.openinfosecfoundation.org/index.html>
- [8] Security Onion Solutions, “Security onion - hardware requirements”, 2026., pristupljeno: 21.5.2026. [Mrežno]. Adresa: <https://docs.securityonion.net/en/2.4/hardware.html>
- [9] Docker Inc., “Docker documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.docker.com>

- [10] InfluxData, “Influxdb 2.x documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/influxdb/v2/>
- [11] —, “Get started with flux”, 2024., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/flux/v0/get-started/>
- [12] —, “Flux query basics”, 2024., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/flux/v0/get-started/query-basics/>
- [13] —, “Flux query language and influxdb basics”, 2024., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/influxdb/v2/query-data/get-started/>
- [14] —, “Telegraf documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/telegraf/v1/>
- [15] —, “Get started with telegraf”, 2025., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/telegraf/v1/get-started/>
- [16] Elastic, “Elasticsearch documentation”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://www.elastic.co/docs/solutions/search>
- [17] —, “Filebeat reference”, 2025., pristupljeno: 29.5.2026. [Mrežno]. Adresa: <https://www.elastic.co/docs/reference/beats/filebeat>
- [18] Grafana Labs, “Grafana documentation”, 2025., pristupljeno: 10.5.2026. [Mrežno]. Adresa: <https://grafana.com/docs/grafana/latest/>
- [19] Y. Rekhter, R. G. Moskowitz, D. Karrenberg, G. J. de Groot, i E. Lear, “Address allocation for private internets”, RFC 1918, Internet Engineering Task Force (IETF), 1996., pristupljeno: 12.6.2026. [Mrežno]. Adresa: <https://www.rfc-editor.org/rfc/rfc1918>
- [20] DeviWiki, “Tp-link tl-wn722n v1.x”, 2018., pristupljeno: 4.6.2026. [Mrežno]. Adresa: [https://deviwiki.com/wiki/TP-LINK\\_TL-WN722N\\_v1.x](https://deviwiki.com/wiki/TP-LINK_TL-WN722N_v1.x)
- [21] Arch Linux Wiki, “Software access point”, 2025., pristupljeno: 7.3.2026. [Mrežno]. Adresa: [https://wiki.archlinux.org/title/Software\\_access\\_point](https://wiki.archlinux.org/title/Software_access_point)

## Skraćenice

|                |                                                          |                                                       |
|----------------|----------------------------------------------------------|-------------------------------------------------------|
| <b>AP</b>      | <i>Access Point</i>                                      | pristupna točka                                       |
| <b>CCMP</b>    | <i>Counter Mode CBC-MAC Protocol</i>                     | protokol šifriranja unutar WPA2                       |
| <b>CPU</b>     | <i>Central Processing Unit</i>                           | središnja procesorska jedinica                        |
| <b>CSV</b>     | <i>Comma-Separated Values</i>                            | vrijednosti odvojene zarezom                          |
| <b>DHCP</b>    | <i>Dynamic Host Configuration Protocol</i>               | protokol za dinamičku dodjelu mrežnih postavki        |
| <b>DNS</b>     | <i>Domain Name System</i>                                | sustav imena domena                                   |
| <b>DoH</b>     | <i>DNS over HTTPS</i>                                    | DNS preko HTTPS-a                                     |
| <b>DoT</b>     | <i>DNS over TLS</i>                                      | DNS preko TLS-a                                       |
| <b>FTP</b>     | <i>File Transfer Protocol</i>                            | protokol za prijenos datoteka                         |
| <b>HTTP</b>    | <i>HyperText Transfer Protocol</i>                       | protokol za prijenos hiperteksta                      |
| <b>IEEE</b>    | <i>Institute of Electrical and Electronics Engineers</i> | Institut inženjera elektrotehnike i elektronike       |
| <b>ILM</b>     | <i>Index Lifecycle Management</i>                        | upravljanje životnim ciklusom indeksa (Elasticsearch) |
| <b>IoC</b>     | <i>Indicator of Compromise</i>                           | indikatori kompromitacije                             |
| <b>IoT</b>     | <i>Internet of Things</i>                                | internet stvari                                       |
| <b>IP</b>      | <i>Internet Protocol</i>                                 | internetski protokol                                  |
| <b>ISP</b>     | <i>Internet Service Provider</i>                         | pružatelj internetskih usluga                         |
| <b>JSON</b>    | <i>JavaScript Object Notation</i>                        | JavaScript objektna notacija                          |
| <b>JVM</b>     | <i>Java Virtual Machine</i>                              | Java virtualni stroj                                  |
| <b>L2 / L3</b> | <i>Layer 2 / Layer 3</i>                                 | podatkovni / mrežni sloj (OSI model)                  |
| <b>LAN</b>     | <i>Local Area Network</i>                                | lokalna mreža                                         |

|               |                                                   |                                                     |
|---------------|---------------------------------------------------|-----------------------------------------------------|
| <b>MAC</b>    | <i>Media Access Control (address)</i>             | adresa upravljanja pristupom mediju                 |
| <b>NAT</b>    | <i>Network Address Translation</i>                | prevođenje mrežnih adresa                           |
| <b>NDJSON</b> | <i>Newline Delimited JSON</i>                     | JSON sa zapisima odvojenima novim retkom            |
| <b>NSM</b>    | <i>Network Security Monitoring</i>                | sigurnosni nadzor mreže                             |
| <b>PSK</b>    | <i>Pre-Shared Key</i>                             | unaprijed dijeljeni ključ                           |
| <b>QUIC</b>   | <i>Quick UDP Internet Connections</i>             | brze UDP internetske veze                           |
| <b>RADIUS</b> | <i>Remote Authentication Dial-In User Service</i> | usluga udaljene autentifikacije korisnika           |
| <b>RAM</b>    | <i>Random Access Memory</i>                       | radna memorija                                      |
| <b>RFC</b>    | <i>Request for Comments</i>                       | niz dokumenata internetskih standarda               |
| <b>SOC</b>    | <i>Security Operations Center</i>                 | sigurnosno-operativni centar                        |
| <b>SQL</b>    | <i>Structured Query Language</i>                  | strukturirani upitni jezik                          |
| <b>SSH</b>    | <i>Secure Shell</i>                               | protokol za sigurnu udaljenu lju-sku                |
| <b>SSID</b>   | <i>Service Set Identifier</i>                     | identifikator bežične mreže                         |
| <b>SSL</b>    | <i>Secure Sockets Layer</i>                       | sloj sigurnih utičnica                              |
| <b>TCP</b>    | <i>Transmission Control Protocol</i>              | protokol za upravljanje prijenosom                  |
| <b>TLS</b>    | <i>Transport Layer Security</i>                   | sigurnost transportnog sloja                        |
| <b>UDP</b>    | <i>User Datagram Protocol</i>                     | korisnički datagramski protokol                     |
| <b>ufw</b>    | <i>Uncomplicated Firewall</i>                     | vatrozid na Linuxu                                  |
| <b>YAML</b>   | <i>YAML Ain't Markup Language</i>                 | jezik za serijalizaciju (konfigura-cijske datoteke) |

# Izjava o korištenju umjetne inteligencije

Prilikom izrade rada korišten je Anthropicov model Claude Sonnet 4.6. Model je korišten za ispravljanje bugova u konfiguracijskim datotekama, generiranje testnih skripti i refaktori-ranje alata kroz razvojni proces kako se rad gradio i kompleksnost povećavala.

Model nije dolazio do zaključaka niti su mu pruženi rezultati testova već kodovi greške s opisom prilikom implementacije i konfiguracije alata odnosno kontejnera. Također UI nije donosila izvršne zaključke rada niti je definirala tok kojim će se rad razvijati već su to radili autor ovog rada i njegov mentor.

## Osvrt na korištenje umjetne inteligencije

Umjetna inteligencija uvelike pomaže pri "dosadnim" zadacima koji su prije bili nužni a sada se daju automatizirati - naravno do neke mjere. UI je uvelike pomogla pri pisanju konfiguracijskih datoteka koje su same po sebi suhoparne sa mnoštvom argumenata koje je potrebno razumijeti jako dobro kako bi sustav funkcionirao. Čak i pri pisanju konfiguracijskih datoteka potrebno je provjeriti izlaz modela jer to početno ponašanje sustava kasnije može diktirati tempo razvoja rada, takozvani tehnički dug. Osim konfiguracijskih skripti, model je pomagao pri pisanju postavljačkih shell skripti kako bi ispis bio bolji i kao provjeru razumnosti autoru. Sve te male stvari uštedjele su mnogo vremena te je osobno mišljenje autora da se umjetnoj inteligenciji treba pristupiti kao alatu koji sposobnim ljudima uvelike povećava efikasnost, a ne služi kao nešto čemu se slijepo vjeruje i na što se treba u potpunosti osloniti.

## Izjava o korištenju umjetne inteligencije

Potvrđujem da sam tijekom izrade ovog diplomskog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.